Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

High-Level Design (HLD)

Мессенджер Kokogram

Для курсовой работы по курсу «Технологии разработки качественного программного обеспечения»

Выполнили Псарев А. С., Никифоров А. В., Соболевский М. А., Зинаков С. С., Ананьев А. М.

Преподаватель Маслаков А. П.

Санкт-Петербург
2026 г.

1. Стек технологий

Kokogram — веб-мессенджер (NF_Platform_1), построенный как набор сервисов: сервис сообщений, сервис чатов, сервис пользователей, сервис онлайн-статусов и сервис GIF. Сервисы обмениваются данными через общие хранилища, брокер событий и синхронные внутренние вызовы по HTTP. Выбранный стек технологий приведён в таблице. Все сервисы разрабатываются на одном языке, Python, что упрощает командную разработку.

1.1. FastAPI (backend)

Все сервисы системы реализуются на Python с использованием фреймворка FastAPI.

1. асинхронная модель подходит сервисам, которые удерживают много открытых соединений и большую часть времени ожидают ответов БД, кэша, брокера и внешнего API;
2. встроенная поддержка WebSocket необходима для доставки событий и онлайн-статусов в реальном времени;
3. валидация входных данных на основе моделей Pydantic закрывает проверки из FS: формат e-mail и телефона, длина и состав пароля, правила для username;
4. спецификация OpenAPI формируется автоматически.
5. механизм внедрения зависимостей позволяет подменять БД, кэш, брокер и внешние клиенты заглушками при тестировании.

Альтернативами выступали Django и Flask. Django ориентирован на синхронные приложения с серверным рендерингом и тяжелее, чем нужно для набора небольших API-сервисов. Во Flask WebSocket и асинхронная работа добавляются сторонними расширениями, а валидация данных и документация API пишутся вручную. FastAPI даёт всё это из коробки, поэтому выбран он.

1.2. PostgreSQL (основная БД)

Данные мессенджера сильно связаны: пользователи состоят в чатах, в чатах хранятся сообщения, пользователи связаны контактами и блокировками. Реляционная модель PostgreSQL позволяет выразить эти связи внешними ключами и сохранять ссылочную целостность.

Сервисы используют один экземпляр PostgreSQL; таблицы разделены по принадлежности сервисам: сервис пользователей отвечает за пользователей, сессии, контакты и блокировки, сервис чатов — за чаты и участников, сервис сообщений — за сообщения, вложения и скрытые сообщения. Требуется PostgreSQL версии 15 или выше: внешний ключ ответов в messages использует ON DELETE SET NULL (reply_to_id) со списком столбцов (раздел 3).

Поскольку данные мессенджера тесно связаны (участники чатов, ответы на сообщения, контакты и блокировки), целостность таких связей, уникальность и проверки ограничениями (UNIQUE, CHECK, внешние ключи) надёжнее обеспечивать на уровне БД, чем в коде. В MySQL нет частичных уникальных индексов (один владелец группы), а SQLite рассчитан на встраивание в один процесс и плохо подходит для набора сервисов с параллельной записью. В PostgreSQL есть и частичные уникальные индексы, и тип jsonb для entities. Поэтому выбрана PostgreSQL.

1.3. Redis (кэш)

Redis хранит данные в оперативной памяти и поддерживает ключи с временем жизни. В системе он используется для трёх задач. Первая — онлайн-статусы: для каждого активного соединения клиента хранится ключ со временем жизни 75 секунд, который продлевается каждым сигналом присутствия; одновременно соединение заносится в отсортированное множество с оценкой, равной моменту истечения ключа, по нему сервис находит просроченные соединения (раздел 4.4); там же хранятся подписки клиентов на статусы, поэтому они не теряются при перезапуске сервиса; пользователь считается Online, пока существует хотя бы один такой ключ, а при обрыве всех соединений ключи исчезают сами, и пользователь считается Offline. Вторая — недавние сообщения чатов: последняя страница истории (30 сообщений) хранится по ключу чата без пользовательских полей; скрытые «у себя» сообщения и статусы «Прочитано» применяются при выдаче, а запись удаляется при отправке, редактировании и удалении сообщений и при удалении чата. Это ускоряет открытие чата. Третья — результаты поиска GIF: кэш помогает уложиться в нефункциональные требования и снижает число обращений к внешнему сервису. Все данные в кэше воспроизводимы из основной БД или внешнего API, поэтому потеря кэша не приводит к потере данных.

1.4. RabbitMQ (брокер событий)

Брокер передаёт события между сервисами: сервис сообщений публикует события сообщений (отправлено, изменено, удалено, прочитано), а сервис чатов — события чатов и участников (чат создан, изменён или удалён, участник добавлен или удалён, роль изменена); по этим событиям сервис онлайн-статусов доставляет данные онлайн-получателям, а сервис сообщений по событию удаления чата очищает файлы и кэш этого чата. RabbitMQ хранит события в очередях до подтверждения обработки, поэтому при кратковременной недоступности сервиса-подписчика события не теряются. Доставка выполняется как минимум один раз, поэтому обработка событий построена идемпотентно (раздел 4.5).

Redis Pub/Sub не хранит сообщения: если сервис-получатель недоступен, событие теряется, а нам нужна доставка с подтверждением обработки. Kafka рассчитан на потоки

данных очень большого объёма и сложнее в развёртывании и сопровождении, чем нужно для нескольких типов событий в нашем проекте. RabbitMQ даёт очереди с подтверждением, маршрутизацию по типу события и простой запуск в Docker.

1.5. SeaweedFS (объектное хранилище)

Файлы из чатов, аватары и превью хранятся в объектном хранилище. В БД остаются только ключи объектов. SeaweedFS — свободное программное обеспечение с лицензией Apache 2.0, поэтому оно бесплатно и запускается в Docker. Хранилище предоставляет S3-совместимый API, и сервисы обращаются к нему через стандартный S3-клиент, поэтому реализацию хранилища можно заменить без изменения кода сервисов. Браузер не может добавить заголовок Authorization к запросу изображения (тег img), поэтому ссылки на файлы, превью и аватары в ответах API — временные подписанные ссылки: к пути эндпоинта добавляются параметры expires (время истечения) и sig (HMAC-SHA256 от пути и expires с секретом, общим для сервисов пользователей, чатов и сообщений). Ссылка формируется при выдаче объекта в ответе API, после проверки права доступа, и действует 15 минут; когда срок истёк, клиент запрашивает сообщение заново (GET /messages/{message_id}) и получает новые ссылки. Gateway пропускает запрос с подписью без проверки токена; подпись и срок проверяет сервис-владелец, после чего отдаёт файл из хранилища. Ключи объектов строятся так, чтобы файлы можно было удалять по префиксу: chats/{chat_id}/{message_id}/... для вложений и превью, avatars/users/{user_id} и avatars/groups/{chat_id} для аватаров. Файлы вложения удаляет сервис сообщений после удаления сообщения, файлы удалённого чата — по событию chat.deleted (раздел 4.5), прежний аватар — сервис, который его заменил.

1.6. Giphy API (внешний GIF-сервис)

Giphy предоставляет каталог GIF с REST API: поиск по текстовому запросу и список трендов. Для разработки доступен бесплатный ключ, число запросов по которому ограничено, поэтому результаты кэшируются в Redis. Ключ API хранится только в сервисе GIF: за списками GIF (поиск, тренды) клиент обращается к нашему серверу, а сами изображения браузер загружает по адресам из ответа — с CDN Giphy. При недоступности Giphy сервис GIF возвращает ошибку, и клиент отображает соответствующее сообщение. Ограничения NF_Gif_2 и NF_Gif_3 обеспечивает сервис GIF: Giphy возвращает для каждой GIF несколько вариантов файла с указанием размера в байтах. Для отправки (url) сервис выбирает самый крупный вариант размером не более 50 МБ, для превью (preview_url) — вариант размером не более 1 МБ (1 МБ = 1 048 576 байт). GIF, для которой подходящего варианта нет, в выдачу не попадает; результат фильтрации кэшируется вместе с результатами поиска. Поэтому клиент не может выбрать GIF, превышающую ограничения. Адрес GIF в запросе отправки клиент мог бы подменить, поэтому сервис сообщений перед сохранением вложения проверяет его через

сервис GIF (GET /internal/gifs/check, раздел 4.6): хост должен входить в список доменов CDN Giphy (https), а размер файла по заголовку Content-Length не превышать 50 МБ. Для адресов, выданных самим сервисом GIF, проверка идёт по кэшу выдачи и не обращается к CDN повторно.

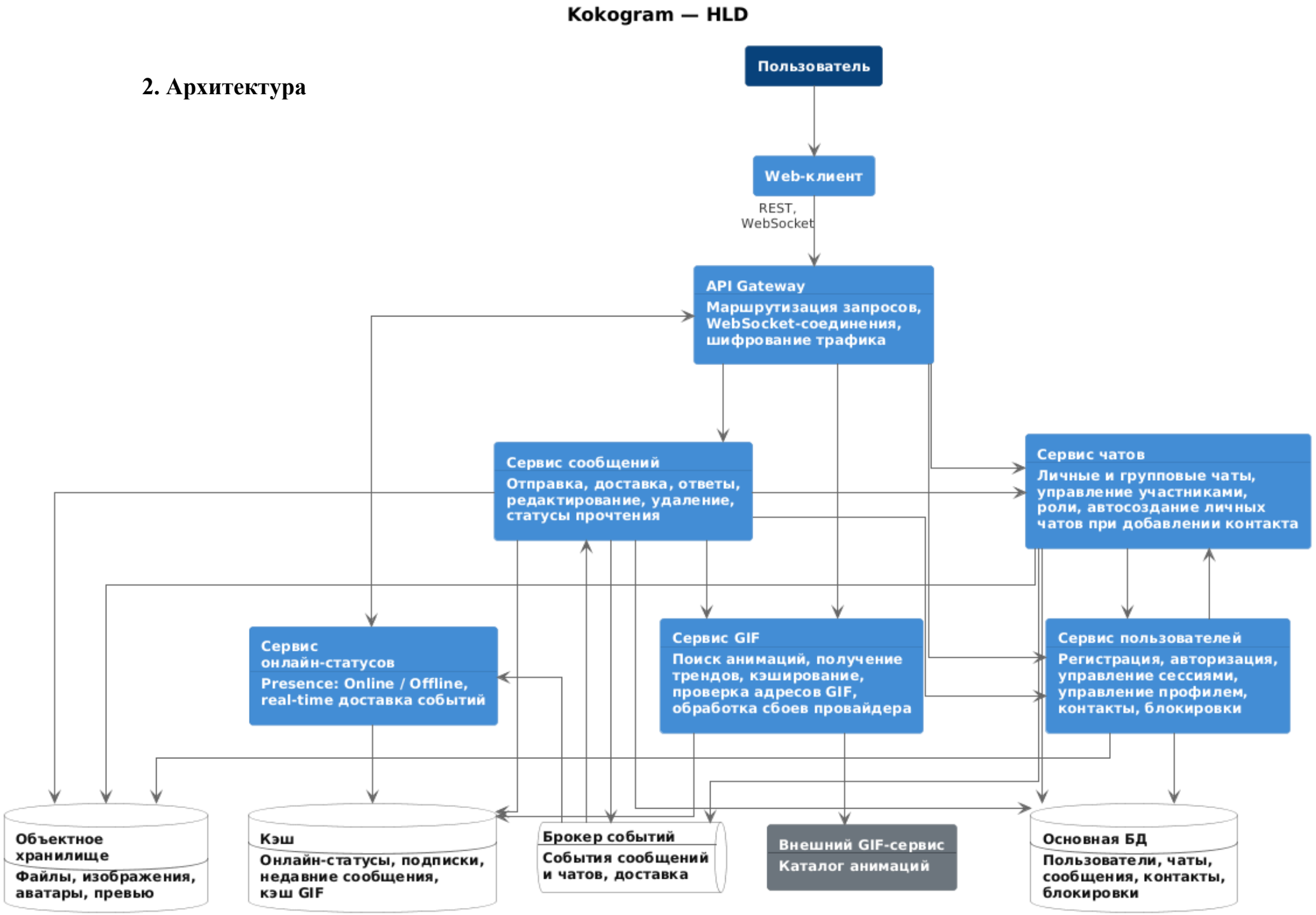
1.7. HTML, CSS, JavaScript (frontend)

Клиент — одностраничное веб-приложение на HTML, CSS и JavaScript без фреймворков и этапа сборки: код подключается ES-модулями, статические файлы раздаёт веб-сервер. Возможностей браузера достаточно для всех экранов мессенджера: fetch для REST-запросов, WebSocket API для событий, DOM для отрисовки. Единственная поддерживаемая платформа — веб-браузер (NF_Platform_1).

1.8. FastAPI и httpx (API Gateway)

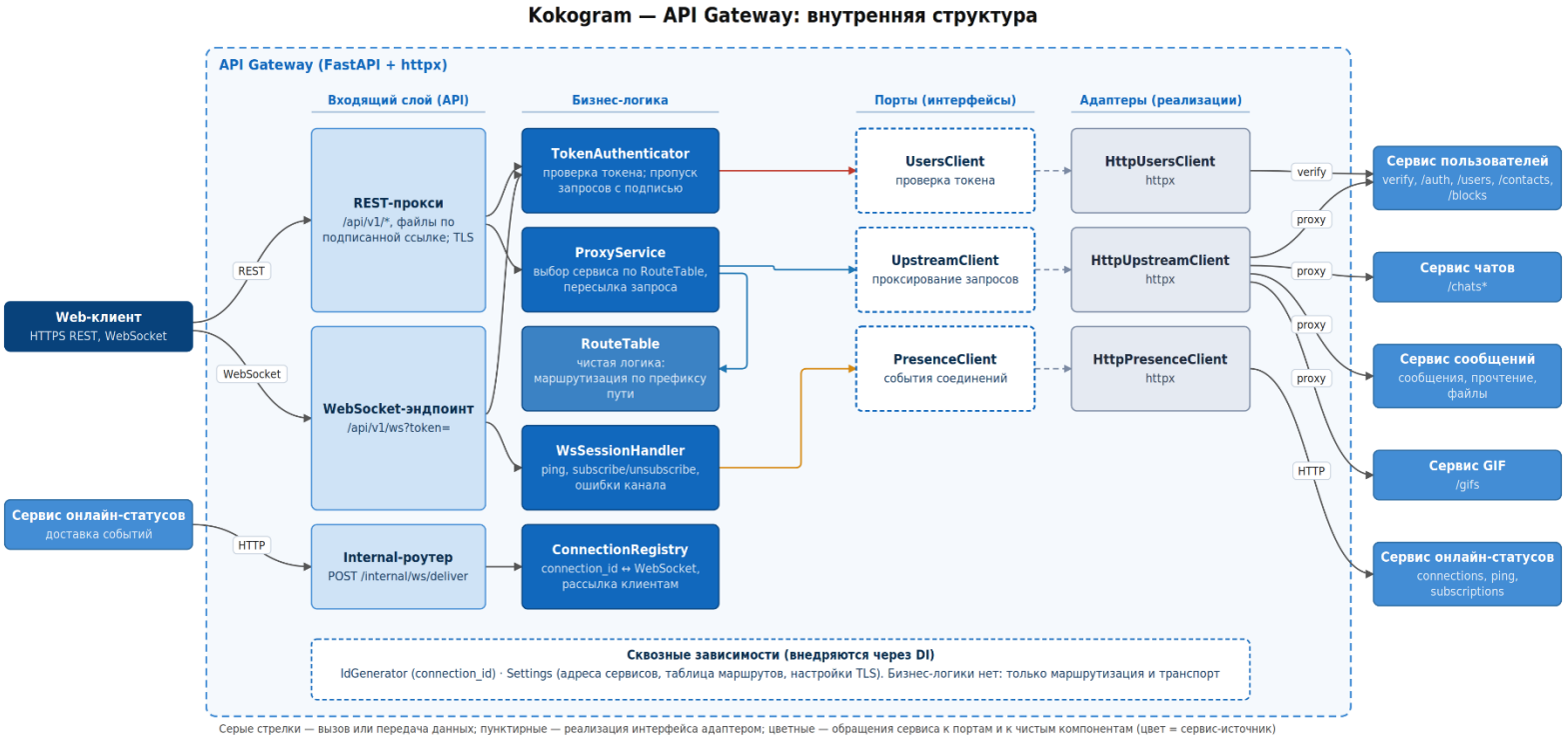
API Gateway реализуется на FastAPI, как и остальные сервисы: тот же язык, тот же набор инструментов тестирования, одинаковая сборка и внедрение зависимостей. Поддерживает WebSocket-соединения и обменивается с сервисом онлайн-статусов внутренними HTTP-вызовами (раздел 4.6), проверяет токен вызовом сервиса пользователей и пропускает запросы файлов с подписью.

2. Архитектура

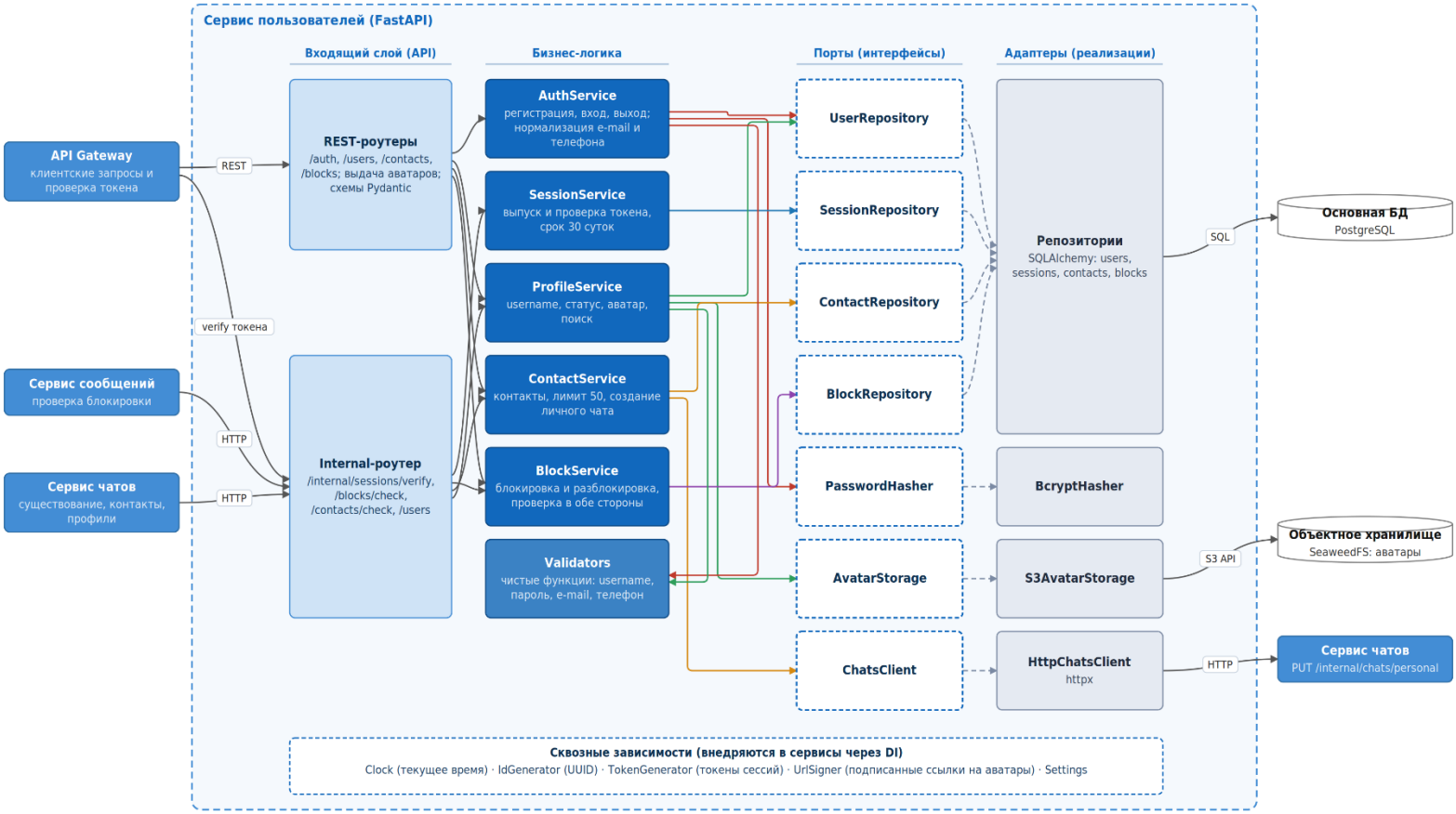


Архитектура Kokogram показана на рисунке выше. Система состоит из веб-клиента, API Gateway, пяти сервисов (пользователей, чатов, сообщений, онлайн-статусов и GIF) и общей инфраструктуры: основной БД, кэша, брокера событий и объектного хранилища. Клиент обращается только к API Gateway. Сервисы вызывают друг друга синхронно по внутренним адресам **/internal** (раздел 4.6) и асинхронно обмениваются событиями через брокер (раздел 4.5); данные каждого сервиса хранятся в принадлежащих ему таблицах (раздел 3). Назначение компонентов приведено в подразделе 2.1, описание связей между ними — в подразделе 2.2.

Схемы каждого модуля отдельно представлены ниже.

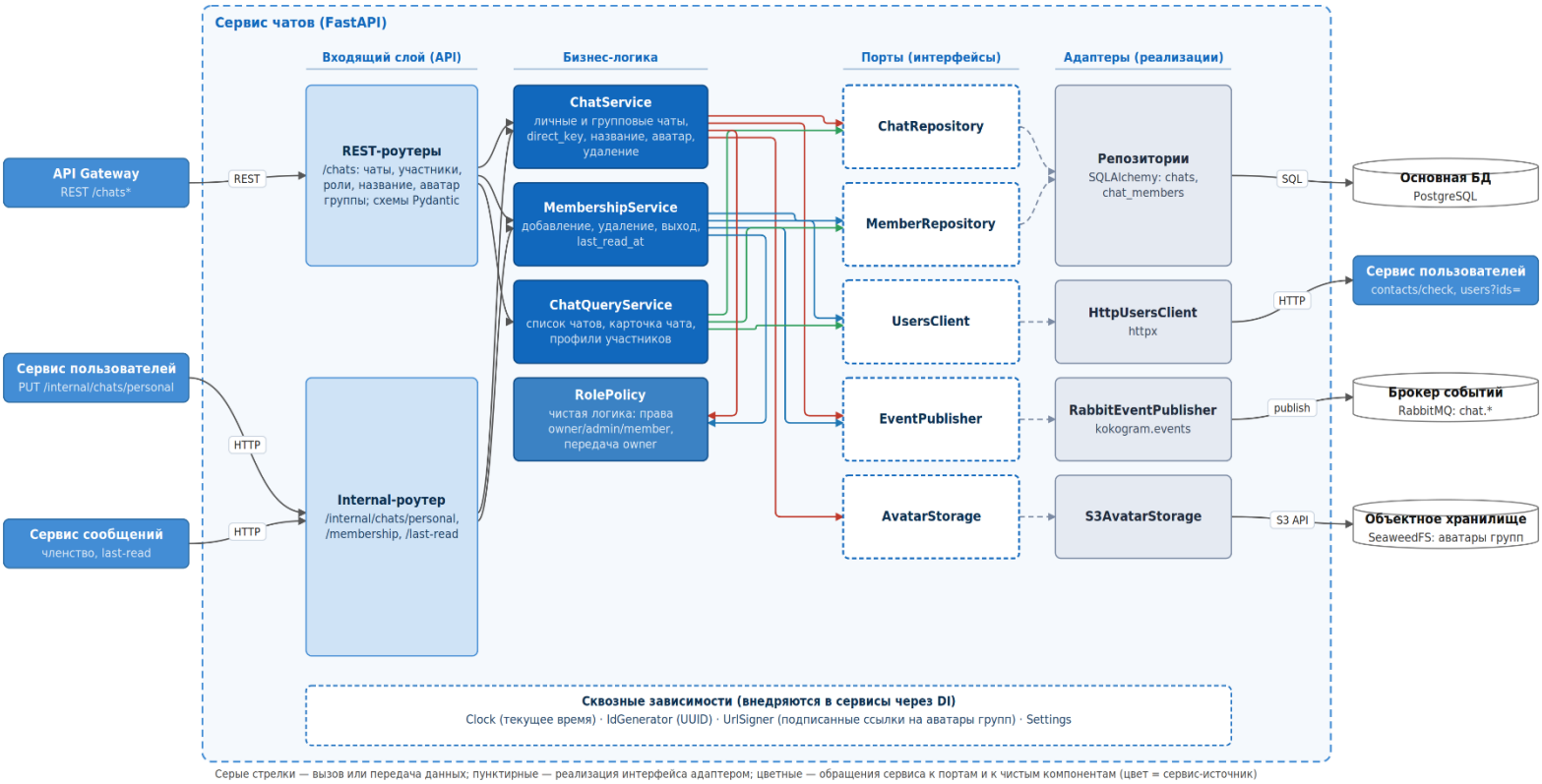


Kokogram — Сервис пользователей: внутренняя структура



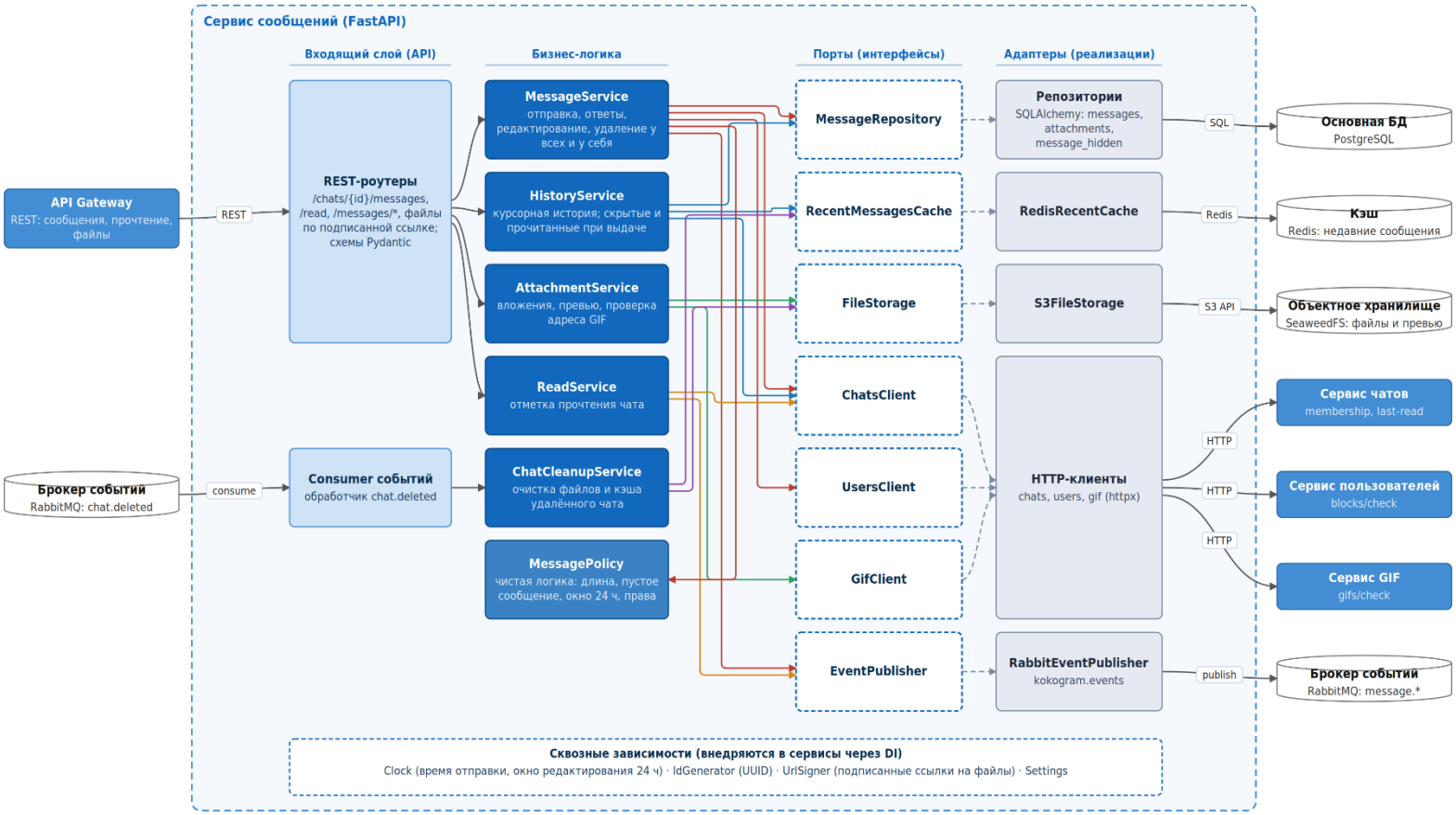
Серые стрелки — вызов или передача данных; пунктирные — реализация интерфейса адаптером; цветные — обращения сервиса к портам и к чистым компонентам (цвет = сервис-источник)

Kokogram — Сервис чатов: внутренняя структура



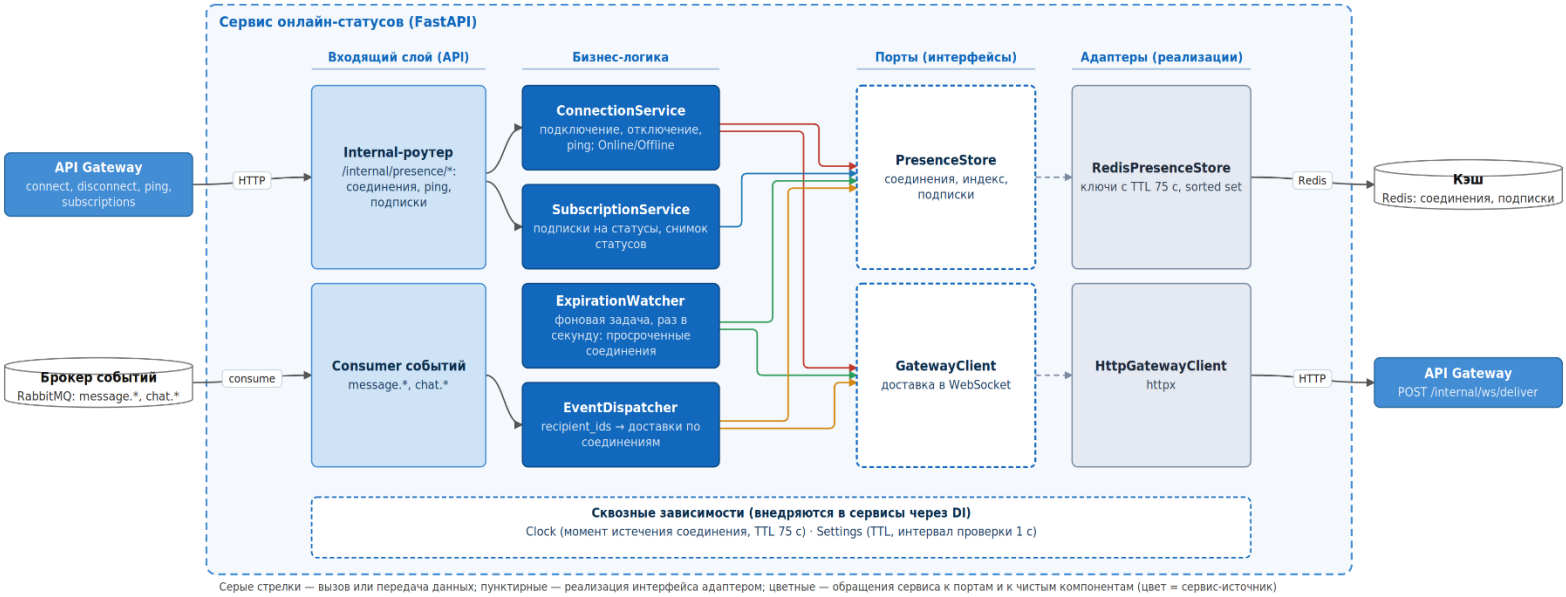
Серые стрелки — вызов или передача данных; пунктирные — реализация интерфейса адаптером; цветные — обращения сервиса к портам и к чистым компонентам (цвет = сервис-источник)

Kokogram — Сервис сообщений: внутренняя структура

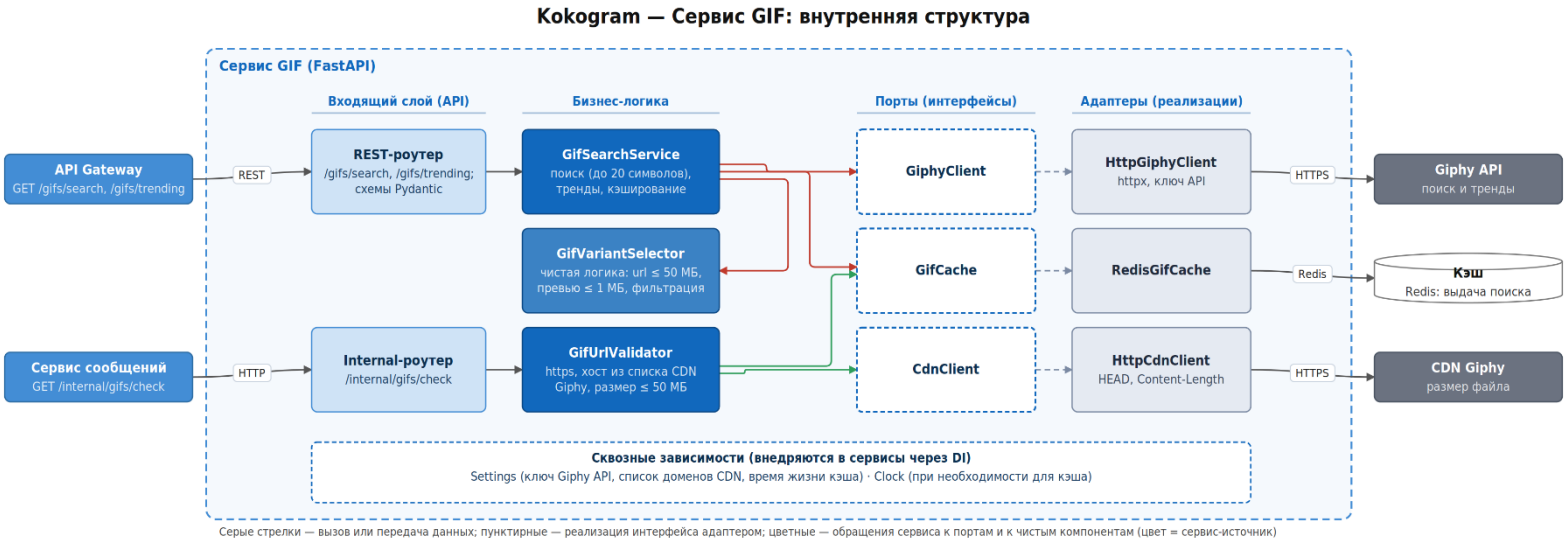


Серые стрелки — вызов или передача данных; пунктирные — реализация интерфейса адаптером; цветные — обращения сервиса к портам и к чистым компонентам (цвет = сервис-источник)

Kokogram — Сервис онлайн-статусов: внутренняя структура



Серые стрелки — вызов или передача данных; пунктирные — реализация интерфейса адаптером; цветные — обращения сервиса к портам и к чистым компонентам (цвет = сервис-источник)



2.1. Описание компонентов

Компонент	Назначение и связи
Пользователь и Web-клиент	Реализуется на HTML, CSS и JavaScript (раздел 1.7). Единственная точка входа пользователя в систему. Взаимодействует только с API Gateway: отправляет REST-запросы (раздел 4.3) и держит одно WebSocket-соединение для событий в реальном времени (раздел 4.4). Токен, выданный при входе, передаёт в заголовке Authorization (REST) или параметром подключения (WebSocket). При подключении подписывается на онлайн-статусы своих контактов и собеседников личных чатов и каждые 25 секунд отправляет ping. Каталог GIF запрашивает у сервера Kokogram, а сами изображения GIF загружает по адресам из ответа (CDN Giphy). Ошибки отображает по кодам из таблицы 4.1.
API Gateway	Реализуется на FastAPI с клиентом httpx (раздел 1.8). Единая внешняя точка входа. Принимает HTTPS-запросы и WebSocket-подключения, обеспечивает шифрование трафика (NF_Sec_1), проверяет токен вызовом POST **/internal/sessions/verify** к сервису пользователей (кроме запросов файлов по подписанной ссылке, раздел 1.5) и маршрутизирует запрос по префиксу пути: **/auth**, **/users**, **/contacts**, **/blocks** — в сервис пользователей; **/chats** (кроме **/chats/{id}/messages** и **/chats/{id}/read**) — в сервис чатов; **/chats/{id}/messages**, **/chats/{id}/read** и **/messages** — в сервис сообщений; **/gifs** — в сервис GIF. Держит WebSocket-соединения клиентов: сообщает сервису онлайн-статусов о подключении и отключении клиента, пересылает ему ping и доставляет в нужные соединения сообщения, которые этот сервис передаёт отдельным внутренним вызовом (разделы 4.4 и 4.6). Адреса **/internal** наружу не маршрутизируются. Бизнес-логики не содержит.
Сервис пользователей	Регистрация и вход, сессии, профиль (username, статус, аватар), поиск по username, список контактов, блокировки. Владеет таблицами users,

Компонент	Назначение и связи
	sessions, contacts и blocks; ограничение в 50 контактов проверяет сам сервис. Аватары пользователей хранит в объектном хранилище. При добавлении контакта синхронно обращается к сервису чатов, который создаёт личный чат (раздел 4.6). Отвечает на синхронные запросы других сервисов: проверка токена (от Gateway), проверка блокировки в обе стороны (от сервиса сообщений, только для личных чатов), проверка существования пользователя и наличия контакта (от сервиса чатов, при добавлении в группу), выдача профилей пользователей по списку идентификаторов (от сервиса чатов). Пароли хэширует алгоритмом bcrypt. События в брокер не публикует. Отдаёт аватары пользователей по подписанной ссылке.
Сервис чатов	Личные и групповые чаты, участники и их роли (owner, admin, member), правила передачи роли owner при выходе владельца, название и аватар группы. Владеет таблицами chats и chat_members, включая last_read_at. Аватар группы хранит в объектном хранилище. По синхронному вызову от сервиса пользователей создаёт личный чат при добавлении контакта, если его ещё нет; повторный вызов безопасен, так как личный чат между парой пользователей уникален по direct_key и возвращается уже существующий. К сервису пользователей обращается при добавлении участника в группу, а также за профилями (username, статус, аватар) участников и собеседников для ответов API; по username выбирает нового владельца при выходе owner. Отвечает сервису сообщений на проверку участника (членство, тип чата, список участников с ролями и last_read_at) и принимает от него момент прочтения чата. При создании и удалении чатов, изменении названия, аватара, состава участников и ролей публикует события *chat.created*, *chat.updated*, *chat.deleted*, *chat.member_added*, *chat.member_removed* и *chat.role_changed* со списком получателей recipient_ids. Отдаёт аватар группы по подписанной ссылке.
Сервис сообщений	Отправка сообщений, история чата с курсорной загрузкой, ответы, редактирование (в течение 24 часов), удаление «у всех» и «у себя», вложения, статусы «Отправлено» и «Прочитано». Владеет таблицами messages, attachments и message_hidden. Файлы и превью изображений хранит в объектном хранилище, недавние сообщения чатов — в кэше. Перед отправкой синхронно проверяет у сервиса чатов, что отправитель состоит в чате, а в личных чатах — у сервиса пользователей, нет ли блокировки. Публикует события *message.created*, *message.edited*, *message.deleted* и *message.read*; список получателей recipient_ids берёт из ответа сервиса чатов. Адрес GIF перед сохранением проверяет у сервиса GIF (раздел 1.6). По событию chat.deleted удаляет из хранилища файлы удалённого чата и сбрасывает его запись в кэше. Отдаёт файлы и превью вложений по подписанной ссылке.
Сервис онлайн-статусов	Определяет онлайн-статус пользователей и доставляет клиентам события в реальном времени. Статус хранится в кэше с временем жизни: ключ существует, пока клиент присылает ping (если сигналов

Компонент	Назначение и связи
	нет более 75 секунд, пользователь считается Offline. Пользователь Online, пока у него есть хотя бы одно активное соединение). Ведёт подписки клиентов на статусы (хранит их в кэше) и по запросу выдаёт их текущее состояние. Потребляет события сообщений и чатов из брокера и передаёт Gateway сообщения для подключённых получателей из recipient_ids. Данные чатов и сообщений не читает. Обмен с Gateway выполняется напрямую внутренними HTTP-вызовами (раздел 4.6), без брокера. Таблиц в основной БД не имеет.
Сервис GIF	Поиск GIF по тексту (до 20 символов) и список популярных GIF. Единственный компонент, который обращается к Giphy API; ключ API хранится только в нём. Результаты поиска кэширует в Redis, чтобы укладываться в нефункциональные требования и сокращать число обращений к внешнему сервису. При недоступности Giphy возвращает ошибку gif_unavailable, по которой клиент показывает сообщение (F_Gif_7). По запросу сервиса сообщений проверяет, что адрес GIF относится к CDN Giphy, а файл не больше 50 МБ (раздел 4.6). Таблиц в основной БД не имеет.
Основная БД	Один экземпляр PostgreSQL; таблицы разделены между сервисами-владельцами (раздел 3). Сервисы не выполняют запросов к чужим таблицам: данные другого сервиса запрашиваются через его API. Хранит ключи объектов, но не сами файлы.
Кэш	Redis. Онлайн-статусы с временем жизни и подписки на статусы (сервис онлайн-статусов), недавние сообщения чатов (сервис сообщений) и результаты поиска GIF (сервис GIF). Все данные воспроизводимы из основной БД или Giphy, поэтому потеря кэша не приводит к потере данных.
Брокер событий	RabbitMQ, обменник *kokogram.events*. Передаёт асинхронные события: *message.created*, *message.edited*, *message.deleted*, *message.read* (из сервиса сообщений в сервис онлайн-статусов) и chat.created, chat.updated, chat.deleted, chat.member_added, chat.member_removed, chat.role_changed (из сервиса чатов в сервис онлайн-статусов; chat.deleted получает также сервис сообщений); состав событий описан в разделе 4.5. События хранятся в очередях до подтверждения обработки.
Объектное хранилище	SeaweedFS с S3-совместимым API. Хранит файлы из чатов, превью изображений, аватары пользователей и аватары групп. Файлы и превью записывает сервис сообщений, аватары пользователей — сервис пользователей, аватары групп — сервис чатов. В основной БД остаются только ключи объектов. Ключи строятся по префиксам чата и пользователя, что позволяет удалять файлы удалённого чата (раздел 1.5).
Внешний GIF-сервис	Giphy API: каталог GIF с поиском по тексту и списком трендов. Каталог доступен только сервису GIF; сами изображения браузер загружает с CDN Giphy по адресам из выдачи. Число запросов по бесплатному ключу ограничено, поэтому результаты кэшируются.

2.2. Описание связей между компонентами

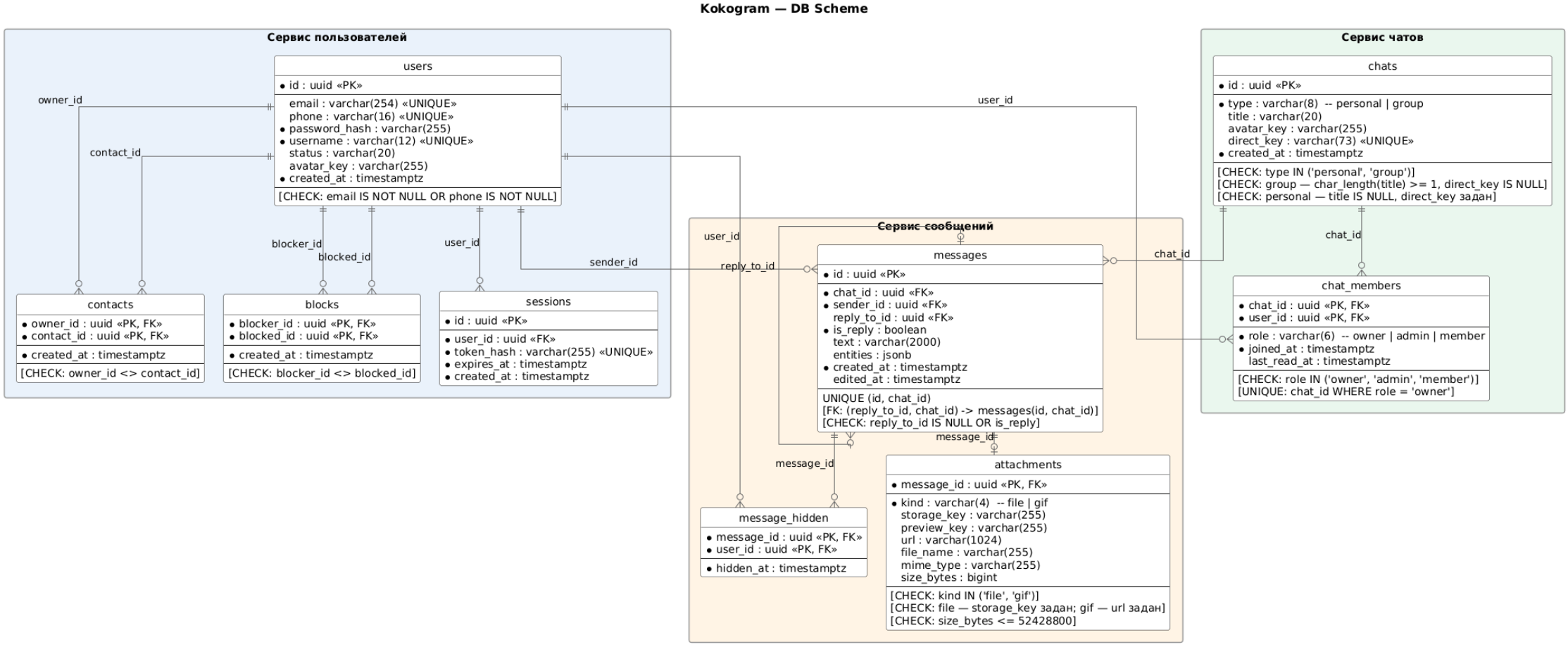
Таблица описывает каждую стрелку схемы: откуда и куда направлена связь, к какому типу она относится и что по ней передаётся. Синхронные вызовы между сервисами подробно описаны в разделе 4.6, события брокера — в разделе 4.5, сообщения WebSocket — в разделе 4.4.

Связь	Тип	Что передаётся и зачем
Пользователь → Web-клиент	Взаимодействие	Пользователь работает с интерфейсом в браузере: вводит данные, читает сообщения, выбирает действия.
Web-клиент → API Gateway	REST, WebSocket	REST: запросы всех операций (раздел 4.3). WebSocket: одно соединение на сессию для получения событий и отправки ping (раздел 4.4). Gateway — единственная точка входа клиента.
API Gateway → Сервис пользователей	Синхронный вызов	Запросы авторизации, профиля, поиска, контактов и блокировок; проверка токена сессии на каждый защищённый запрос (раздел 4.6).
API Gateway → Сервис чатов	Синхронный вызов	Запросы управления чатами: создание, участники, роли, название и аватар группы.
API Gateway → Сервис сообщений	Синхронный вызов	Отправка, история, редактирование и удаление сообщений, отметка прочтения, скачивание вложений.
API Gateway → Сервис GIF	Синхронный вызов	Поиск GIF по тексту и список популярных GIF.
API Gateway ↔ Сервис онлайн-статусов	Синхронные вызовы в обе стороны	К сервису: подключение и отключение клиента, ping, подписки на статусы. От сервиса: вызов доставки событий (статусы, сообщения, изменения чатов), которые Gateway передаёт в нужные WebSocket-соединения. Брокер в этом обмене не участвует (разделы 4.4 и 4.6).
Сервис сообщений → Сервис чатов	Синхронный вызов	Проверка, что отправитель состоит в чате; тип чата; список участников с ролями и last_read_at (получатели событий); запись момента прочтения (раздел 4.6).
Сервис сообщений → Сервис пользователей	Синхронный вызов	Проверка блокировки между собеседниками (только для личных чатов).
Сервис сообщений → Сервис GIF	Синхронный вызов	Проверка адреса GIF перед сохранением вложения: хост CDN Giphy и размер не более 50 МБ (раздел 4.6).

Связь	Тип	Что передаётся и зачем
Сервис чатов → Сервис пользователей	Синхронный вызов	Проверка существования пользователя и наличия его в контактах при добавлении в группу; получение профилей участников и собеседников для ответов API.
Сервис пользователей → Сервис чатов	Синхронный вызов	Создание личного чата при добавлении контакта; без успешного ответа контакт не сохраняется (раздел 4.6).
Сервис сообщений → Брокер событий	Событие	message.created, message.edited, message.deleted, message.read со списком получателей recipient_ids.
Сервис чатов → Брокер событий	Событие	chat.created, chat.updated, chat.deleted, chat.member_added, chat.member_removed, chat.role_changed со списком получателей recipient_ids.
Брокер событий → Сервис онлайн-статусов	Событие	События сообщений и чатов: сервис находит онлайн-получателей и передаёт данные Gateway для доставки по WebSocket.
Брокер событий → Сервис сообщений	Событие	chat.deleted: сервис сообщений удаляет файлы удалённого чата из хранилища и сбрасывает его запись в кэше.
Сервис пользователей → Основная БД	Чтение и запись	Таблицы users, sessions, contacts, blocks.
Сервис чатов → Основная БД	Чтение и запись	Таблицы chats и chat_members.
Сервис сообщений → Основная БД	Чтение и запись	Таблицы messages, attachments и message_hidden.
Сервис пользователей → Объектное хранилище	Запись и чтение файлов	Аватары пользователей.
Сервис чатов → Объектное хранилище	Запись и чтение файлов	Аватары групповых чатов.
Сервис сообщений → Объектное хранилище	Запись и чтение файлов	Файлы вложений и превью изображений: запись, чтение и удаление (в том числе по префиксу удалённого чата).
Сервис сообщений → Кэш	Чтение и запись	Недавние сообщения чатов: последняя страница истории; запись сбрасывается при изменении сообщений чата.
Сервис онлайн-статусов → Кэш	Чтение и запись	Ключи соединений со временем жизни и множество моментов их истечения (по ним определяется статус Online/Offline), а также подписки на статусы.

Связь	Тип	Что передаётся и зачем
Сервис GIF → Кэш	Чтение и запись	Результаты поиска и список популярных GIF со временем жизни.
Сервис GIF → Внешний GIF-сервис	HTTPS, REST	Запросы поиска и трендов к Giphy API.

3. Схема базы данных



Основная БД Kokogram — PostgreSQL. Схема приведена на рисунке выше. Таблицы сгруппированы по сервисам, которые ими владеют: сервис пользователей отвечает за пользователей, сессии, контакты и блокировки; сервис чатов — за чаты и их участников; сервис сообщений — за сообщения, вложения и скрытие сообщений. Внешние ключи обеспечивают целостность данных между таблицами разных сервисов, но прямых запросов к чужим таблицам сервисы не выполняют: данные другого сервиса запрашиваются через его API (например, проверка участника чата). Бизнес-правила каждого сервиса вынесены в заметки на схеме.

Идентификаторы всех сущностей — UUID, время хранится в типе timestamptz. В основной БД не хранятся онлайн-статусы и результаты поиска GIF (они находятся в Redis), а также содержимое файлов и аватаров: в таблицах лежат только ключи объектов в объектном хранилище.

3.1. Описание таблиц, ограничений целостности и индексов

Таблица	Назначение и особенности
users	Учётные записи и профили. При регистрации указываются username, пароль и e-mail или телефон (достаточно одного из двух), поэтому каждое из полей email и phone может быть пустым, но хотя бы одно заполнено (ограничение CHECK). Телефон хранится в нормализованном виде (+7XXXXXXXXXX), e-mail — в нижнем регистре; нормализацию выполняет сервис пользователей до записи в БД. Пароль хранится только в виде хэша bcrypt со стоимостью 10 (NF_Sec_2). username вводится пользователем при регистрации, может быть изменён позже и используется для поиска. status — текстовый статус профиля (до 20 символов), avatar_key — ключ аватара в объектном хранилище. **Ограничения целостности.** Уникальность email, phone и username обеспечивается ограничениями UNIQUE на уровне БД, поэтому дубликаты невозможны даже при одновременных запросах. **Индексы.** Поиск по username использует уникальный индекс.
sessions	Сессии входа. При входе создаётся запись: token_hash — SHA-256 от выданного токена (токен — 32 случайных байта, сам токен в БД не хранится (NF_Sec_4)), expires_at — срок действия сессии (30 суток с момента входа). У пользователя может быть несколько сессий. Выход из аккаунта или истечение срока — удаление записи.
contacts	Список контактов: пара (owner_id, contact_id). Связь односторонняя: добавленный контакт не обязан иметь владельца в своём списке. Ограничение в 50 контактов проверяет сервис пользователей. **Ограничения целостности.** Ограничение CHECK запрещает добавлять в контакты самого себя (owner_id <> contact_id).

Таблица	Назначение и особенности
blocks	Блокировки: пара (blocker_id, blocked_id). Пока запись существует, обмен сообщениями в личном чате между этими пользователями запрещён в обе стороны; в групповых чатах блокировка не действует. Блокировка не удаляет контакт и не запрещает добавлять заблокированного пользователя в контакты. Разблокировка — удаление записи. **Ограничения целостности.** Ограничение CHECK запрещает блокировать самого себя (blocker_id <> blocked_id). **Индексы.** Индекс по blocked_id нужен для проверки обратной блокировки.
chats	Чаты. Поле type принимает значения personal и group. title (от 1 до 20 символов) и avatar_key относятся только к групповым чатам, у личных чатов title пуст. direct_key — ключ пары участников личного чата (меньший и больший id пользователей через разделитель); он уникален и гарантирует, что у двух пользователей существует только один личный чат. У групповых чатов равен NULL. **Ограничения целостности.** Уникальность direct_key обеспечивается ограничением UNIQUE на уровне БД, поэтому второй личный чат между той же парой пользователей невозможен даже при одновременных запросах. Ограничения CHECK: у группового чата название не пусто, а direct_key пуст; у личного чата название пусто, а direct_key задан. Допустимые значения type (personal, group) также задаются ограничением CHECK. **Каскадное удаление.** Удаление чата удаляет его участников и сообщения; файлы удалённого чата из хранилища удаляет сервис сообщений по событию chat.deleted (раздел 4.5).
chat_members	Участники чатов и их роли (owner, admin, member). last_read_at — момент, до которого участник прочитал чат (NULL — чат ещё не прочитан); по нему определяется статус «Прочитано». Выход из группы и удаление участника — удаление записи. В личном чате ровно два участника — те, чьи идентификаторы указаны в direct_key, оба с ролью member. **Ограничения целостности.** Один владелец (owner) на групповой чат гарантируется частичным уникальным индексом по chat_id для записей с role = owner. Наличие владельца у группы БД не гарантирует, это обеспечивает сервис чатов: создатель группы получает роль owner, а при выходе владельца роль передаётся другому участнику в одной транзакции с удалением его записи, чтобы группа не осталась без owner. Допустимые значения role (owner, admin, member) задаются ограничением CHECK. **Индексы.** Индекс по user_id нужен для получения списка чатов пользователя. **Каскадное удаление.** Записи участников удаляются вместе с чатом.

Таблица	Назначение и особенности
messages	Сообщения. text — до 2000 символов (может быть пустым, если в сообщении только вложение; сообщение без текста и без вложения не допускается). entities — диапазоны форматирования текста (жирный, курсив, зачёркнутый) в формате JSON; эмодзи хранятся непосредственно в тексте. reply_to_id — ссылка на исходное сообщение при ответе. edited_at — время последнего редактирования: если оно задано, у сообщения отображается приписка «edited». **Ответы внутри одного чата.** Сообщение-ответ должно ссылаться на сообщение того же чата. Для этого в messages задан уникальный ключ (id, chat_id), а внешний ключ ответа ссылается на пару (reply_to_id, chat_id), поэтому ответ на сообщение из другого чата невозможен на уровне БД. **Индексы.** Индекс (chat_id, created_at) используется для загрузки истории чата. **Каскадное удаление.** Сообщения удаляются вместе с чатом. Удаление сообщения удаляет его вложение (файл из хранилища сервис сообщений удаляет после удаления записи) и записи в message_hidden, а у ответов на это сообщение поле reply_to_id становится NULL (внешний ключ задан как ON DELETE SET NULL (reply_to_id), чтобы не обнулялся chat_id; нужен PostgreSQL 15 или выше).
attachments	Вложение сообщения (не более одного). Для kind = file заполняются storage_key, file_name, mime_type (тип содержимого файла, например image/png) и size_bytes (файл лежит в объектном хранилище), а для изображений — ещё и preview_key (ключ превью); для kind = gif заполняется url (GIF из Giphy; адрес проверяется при отправке, раздел 1.6). **Ограничения целостности.** Ограничения CHECK: у файла задан storage_key, у GIF задан url; размер файла не больше 52 428 800 байт (50 МБ при 1 МБ = 1 048 576 байт). Допустимые значения kind (file, gif) также задаются ограничением CHECK. **Каскадное удаление.** Вложение удаляется вместе с сообщением.
message_hidden	Сообщения, которые пользователь удалил только у себя: пара (message_id, user_id). **Каскадное удаление.** Записи удаляются вместе с сообщением.

4. Спецификация API

Раздел описывает внешний и внутренний интерфейсы системы: REST API, доступный клиенту через API Gateway; WebSocket-канал для событий в реальном времени; события, которыми сервисы обмениваются через брокер; синхронные вызовы между сервисами. Для понимания связности в таблицах указаны идентификаторы требований FS.

4.1. Общие соглашения

Адреса и формат. REST-запросы выполняются по HTTPS (NF_Sec_1) и начинаются с префикса **/api/v1**. Тела запросов и ответов — JSON, кроме загрузки файлов и скачивания вложений. Идентификаторы — UUID, время — ISO 8601 в UTC.

Авторизация. Все запросы, кроме POST **/auth/register** и POST **/auth/login**, содержат заголовок *Authorization: Bearer <token>*. Токен выдаётся при входе, сервер хранит только его хэш. Если токен отсутствует, просрочен или сессия завершена, возвращается *401 unauthorized*. Исключение — запросы файлов по подписанной ссылке (раздел 1.5): вместо заголовка они содержат параметры expires и sig.

Формат ошибок. Все сервисы возвращают ошибку в одной форме: **{"error": {"code": "...", "message": "...", "details": [...]}}**. Поле *code* — определенный машинный код (по нему клиент выбирает поведение, а тесты проверяют результат), *message* — текст для пользователя, *details* — необязательные подробности, например список полей с причинами. Коды и соответствующие HTTP-статусы приведены в таблице.

Постраничная загрузка. История чата загружается по курсору: параметр *limit* (по умолчанию 30, не более 100) и параметр *before* — значение *next_cursor* из предыдущего ответа. Ответ имеет вид **{"items": [...], "next_cursor": "..." или null}**, сообщения идут от новых к старым.

Код ошибки	HTTP	Когда возвращается	FS
validation_error	422	Нарушены правила формата или длины. В details указаны поля и причины: invalid_email, invalid_phone, password_length, password_composition, username_required, username_format, username_length, status_length, avatar_format, title_length, text_too_long, message_empty, query_too_long, self_target, gif_invalid	F_Auth_3, 5, 7, 8, 12; F_Prof_2, 3, 6, 9; F_Chat_3; F_Group_17, 20; F_Gif_3, 4
email_taken	409	Адрес электронной почты уже зарегистрирован	F_Auth_4
phone_taken	409	Номер телефона уже зарегистрирован	F_Auth_6

Код ошибки	HTTP	Когда возвращается	FS
username_taken	409	Username уже используется другим пользователем	F_Prof_4
invalid_credentials	401	Неверный пароль или логин при входе	F_Auth_11
unauthorized	401	Токен отсутствует, сессия просрочена или завершена	NF_Sec_3
forbidden	403	Недостаточно прав для действия (роль в чате; действие над чужим сообщением)	F_Group_5–11; F_Chat_8, 11, 12
not_member	403	Пользователь не является участником чата	—
blocked	403	Отправка сообщения в личном чате при блокировке в любую сторону	F_Block_2, 3
edit_window_expired	403	С момента отправки сообщения прошло более 24 часов	F_Chat_10
not_a_contact	422	Добавляемого в группу пользователя нет в контактах добавляющего	F_Group_19
contact_limit_reached	409	В списке контактов уже 50 пользователей	F_Cont_2
already_member	409	Пользователь уже состоит в чате	F_Group_23
not_found	404	Объект не найден или недоступен пользователю	—
file_too_large	413	Файл вложения больше 50 МБ или аватар больше 5 МБ	F_File_2, F_Prof_7
gif_unavailable	503	Внешний GIF-сервис недоступен	F_Gif_7
service_unavailable	503	Внутренний сервис, необходимый для операции, недоступен (например, сервис чатов при добавлении контакта или сервис пользователей при выдаче списка участников)	—

4.2. Основные объекты

Объекты, которые передаются в ответах REST API и в событиях WebSocket. Поля с пометкой «или null» могут отсутствовать.

Объект	Поля
User	id; username; status (или null); avatar_url (подписанная ссылка, раздел 1.5, или null). В ответах /users/me дополнительно email и phone (или null). Онлайн-статус в объект не входит и приходит по WebSocket.
Chat	id; type (personal или group); title (для группы); avatar_url (для группы; подписанная ссылка, или null); peer — объект User собеседника (для личного чата); my_role (owner, admin или member); created_at.
ChatMember	user (объект User); role (owner, admin или member); joined_at.
Message	id; chat_id; sender_id; text (или null); entities — список фрагментов форматирования вида {type: bold, italic или strike; offset; length}; reply_to — null, если сообщение не ответ, объект {message_id, sender_id, text_preview} для ответа или {deleted: true}, если исходное сообщение удалено; attachment (или null); status (sent или read, для сообщений текущего пользователя); created_at; edited_at (или null, при наличии у сообщения отображается приписка «edited»).

Объект	Поля
Attachment	kind (file или gif); file_name, mime_type, size_bytes; url — подписанная ссылка на скачивание файла (раздел 1.5) или адрес GIF; preview_url — подписанная ссылка на превью (для изображений, или null).
Gif	id; url — адрес GIF для отправки (вариант размером не более 50 МБ, NF_Gif_3); preview_url — адрес превью (вариант размером не более 1 МБ, NF_Gif_2).

4.3. REST API

Эндпоинты сгруппированы по сервисам-владельцам. API Gateway маршрутизирует запросы по префиксу пути (далее в пункте 4.6). В столбце «Ошибки» перечислены коды из таблицы выше, характерные для эндпоинта; во всех эндпоинтах, кроме регистрации и входа, возможна ошибка unauthorized.

Сервис пользователей (авторизация)

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
POST /auth/register	Регистрация. Тело: username, password, email или phone.	201, объект User	validation_error, email_taken, phone_taken, username_taken	F_Auth_1–8, 12–14; F_Prof_2–4; NF_Perf_1
POST /auth/login	Вход. Тело: login (адрес электронной почты или номер телефона), password. Создаёт сессию; допускается несколько сессий одновременно.	200, {token, expires_at, user}	invalid_credentials	F_Auth_9–11, 13, 14; NF_Perf_2; NF_Sec_3
POST /auth/logout	Выход из аккаунта: завершает текущую сессию (остальные сессии пользователя не затрагиваются).	204	unauthorized	—

Сервис пользователей (профиль и поиск)

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
GET /users/me	Профиль текущего пользователя.	200, объект User	—	—
PATCH /users/me	Изменение профиля. Тело: username и/или status.	200, объект User	validation_error, username_taken	F_Prof_1–4, 8, 9
PUT /users/me/avatar	Загрузка аватара. Multipart, поле file (только jpg и png, до 5 МБ).	200, объект User	validation_error, file_too_large	F_Prof_5–7

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
GET /users/{user_id}	Профиль пользователя (без email и phone).	200, объект User	not_found	—
GET /users/{user_id}/avatar	Аватар пользователя. Доступ по подписанной ссылке из avatar_url (раздел 1.5).	200, содержимое файла	unauthorized (неверная или просроченная подпись), not_found	F_Prof_5
GET /users?username=	Поиск пользователя по username. Возвращает результат только при полном совпадении.	200, {items: [User]} (ноль или один элемент)	validation_error	F_Search_1, 2; NF_Perf_3

Сервис пользователей (контакты и блокировки)

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
GET /contacts	Список контактов текущего пользователя.	200, {items: [User]}	—	F_Cont_4; F_Group_3
POST /contacts	Добавление в контакты. Тело: user_id. Перед сохранением контакта сервис синхронно обращается к сервису чатов, который создаёт личный чат (раздел 4.6); ответ 201 означает, что личный чат существует. Если сервис чатов недоступен, контакт не сохраняется. Блокировка пользователя не мешает добавить его в контакты.	201, объект User	not_found, validation_error (self_target), contact_limit_reached, service_unavailable	F_Cont_1–3; F_Block_7
GET /blocks	Список пользователей, заблокированных текущим пользователем.	200, {items: [User]}	—	F_Block_4
PUT /blocks/{user_id}	Блокировка пользователя. Повторный вызов результата не меняет. Контакт не удаляется, а добавлять заблокированного пользователя в контакты по-прежнему можно.	204	not_found, validation_error (self_target)	F_Block_1–3, 6
DELETE /blocks/{user_id}	Разблокировка пользователя.	204	not_found	F_Block_4, 5

Сервис чатов

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
GET /chats	Список чатов текущего пользователя.	200, {items: [Chat]}	—	F_Group_12, 14

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
POST /chats/personal	Открыть диалог с пользователем: возвращает существующий личный чат или создаёт его. Тело: user_id. Контакт для этого не нужен. Если личный чат одновременно создан другим запросом (в том числе при добавлении контакта), возвращается уже существующий чат (уникальность direct_key).	200 (чат есть) или 201 (создан), объект Chat	not_found	F_Chat_17, 18
POST /chats/groups	Создание группового чата. Тело: title (от 1 до 20 символов). Создатель получает роль owner.	201, объект Chat	validation_error (title_length)	F_Group_1, 2, 17
GET /chats/{chat_id}	Сведения о чате и роль текущего пользователя.	200, объект Chat	not_found, not_member	—
PATCH /chats/{chat_id}	Изменение названия группы. Тело: title. Доступно owner и admin.	200, объект Chat	validation_error, forbidden	F_Group_7, 20
PUT /chats/{chat_id}/avatar	Изменение аватара группы. Multipart, поле file. Доступно owner и admin.	200, объект Chat	forbidden, validation_error, file_too_large	F_Group_8
GET /chats/{chat_id}/avatar	Аватар группы. Доступ по подписанной ссылке из avatar_url (раздел 1.5).	200, содержимое файла	unauthorized (неверная или просроченная подпись), not_found	F_Group_8
DELETE /chats/{chat_id}	Удаление группового чата. Доступно только owner. Чат исчезает у всех участников.	204	forbidden, not_found	F_Group_11, 12
GET /chats/{chat_id}/members	Список участников чата с ролями.	200, {items: [ChatMember]}	not_member	—
POST /chats/{chat_id}/members	Добавление участника в группу. Тело: user_id. Доступно любому участнику; добавляемый должен быть в контактах добавляющего, получает роль member и видит всю историю чата.	201, объект ChatMember	not_member, not_found, not_a_contact, already_member	F_Group_3, 4, 19, 21, 23
PATCH /chats/{chat_id}/members/{user_id}	Изменение роли участника. Тело: role (admin или member). Доступно только owner.	200, объект ChatMember	forbidden, not_found	F_Group_5, 6
DELETE /chats/{chat_id}/members/{user_id}	Удаление участника из группы: owner удаляет admin и member, admin — только member.	204	forbidden, not_found	F_Group_9, 10

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
DELETE /chats/{chat_id}/members/me	Выход из группы. Если выходит owner, роль переходит к первому по алфавиту (по username) admin, а при их отсутствии — member; передача роли и удаление записи вышедшего выполняются в одной транзакции, чтобы группа не осталась без owner. Если вышел последний участник, чат удаляется.	204	not_member	F_Group_13–16, 18

Сервис сообщений

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
GET /chats/{chat_id}/messages	История чата. Параметры: limit, before (курсор). Включает сообщения, отправленные до вступления пользователя в группу; не включает сообщения, удалённые пользователем «у себя».	200, {items: [Message], next_cursor}	not_member	F_Chat_4, 5, 7, 9; F_Group_21, 24
POST /chats/{chat_id}/messages	Отправка сообщения. JSON: {text, entities, reply_to_id, gif_url}; для файла — multipart с полем file и теми же полями. Нужен text или вложение. gif_url — адрес из выдачи сервиса GIF: перед сохранением сервис сообщений проверяет через сервис GIF, что хост относится к CDN Giphy, а размер не более 50 МБ (раздел 1.6). Публикует событие message.created.	201, объект Message	validation_error (в том числе gif_invalid), file_too_large, not_member, blocked, not_found (ответ на несуществующее сообщение), gif_unavailable, service_unavailable	F_Chat_2, 3, 6, 13–16; F_File_1, 2; F_Gif_4; F_Block_2, 3; NF_Gif_3
PATCH /messages/{message_id}	Редактирование своего сообщения не позднее чем через 24 часа после отправки. Тело: text, entities. Публикует message.edited.	200, объект Message	forbidden, edit_window_expired, validation_error, not_found	F_Chat_8–10
DELETE /messages/{message_id}?scope=	Удаление своего сообщения. scope=all — у всех участников (публикует message.deleted), scope=me — только у себя.	204	forbidden, not_found	F_Chat_11, 12
POST /chats/{chat_id}/read	Пометить чат прочитанным до текущего момента (записывает last_read_at). Публикует message.read.	204	not_member	F_Chat_5; F_Group_24
GET /messages/{message_id}/attachment	Скачивание файла вложения; с параметром preview=true — превью изображения. Доступ по подписанной ссылке из url и preview_url (раздел 1.5) или с заголовком Authorization.	200, содержимое файла	unauthorized (неверная или просроченная подпись), not_member, not_found	F_File_1, 3

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
GET /messages/{message_id}	Получить сообщение целиком; вложение в ответе содержит новые подписанные ссылки. Клиент вызывает, когда срок действия ссылки истёк (раздел 1.5).	200, объект Message	not_member, not_found	F_File_1, 3

Сервис GIF

Метод и путь	Назначение, параметры	Успешный ответ	Ошибки	FS
GET /gifs/search?q=	Поиск GIF по тексту (до 20 символов). Если ничего не найдено, возвращается пустой список, и клиент показывает соответствующее сообщение. GIF, не удовлетворяющие NF_Gif_2 и NF_Gif_3, в выдачу не включаются (раздел 1.6).	200, {items: [Gif]}	validation_error (query_too_long), gif_unavailable	F_Gif_1–3, 6, 7; NF_Gif_1–3
GET /gifs/trending	Популярные GIF (используется при пустом поисковом запросе). GIF, не удовлетворяющие NF_Gif_2 и NF_Gif_3, в выдачу не включаются (раздел 1.6).	200, {items: [Gif]}	gif_unavailable	F_Gif_5, 7; NF_Gif_1–3

4.4. WebSocket-канал

Канал нужен для событий, которые сервер сам отправляет клиенту: новые и изменённые сообщения, статусы «Прочитано», онлайн-статусы контактов, изменения чатов и состава участников. Пользователь считается Online, пока у него есть хотя бы одно активное соединение (одновременно возможны несколько сессий). Клиент подключается по адресу **wss://{host}/api/v1/ws?token=<token>**: браузерный WebSocket не позволяет задать заголовок Authorization, поэтому токен передаётся параметром. При недействительном токене соединение закрывается с кодом 4401. Подключение и отключение клиента определяют его онлайн-статус.

Все сообщения канала — JSON вида **{"type": "...", "payload": {...}}**.

Сообщения клиента серверу

Тип	Содержимое	Назначение
subscribe_status	{user_ids: [uuid]}	Подписаться на онлайн-статусы пользователей: клиент отправляет при подключении список своих контактов и собеседников личных чатов и добавляет пользователя, когда открывает его профиль или личный чат. В ответ приходит status_snapshot.
unsubscribe_status	{user_ids: [uuid]}	Отписаться от статусов (профиль закрыт, контакт удалён).
ping	{}	Сигнал присутствия: отправляется примерно каждые 25 секунд. Если сигналов нет более 75 секунд, соединение считается потерянным, и пользователь получает статус Offline, если у него нет других активных соединений.

Сообщения сервера клиенту

Тип	Содержимое	Когда отправляется
status_snapshot	{statuses: [{user_id, online}]}	В ответ на subscribe_status: текущие статусы, которые Сервис онлайн-статусов берёт из кэша (F_Pres_1, 2; F_Chat_1).
status_changed	{user_id, online}	Статус пользователя, на которого подписан клиент, изменился (у пользователя появилось первое активное соединение или пропало последнее); доставка не позднее 2 секунд (NF_Perf_4; F_Chat_1).
message_created	{message: Message}	В чат, участником которого является пользователь, отправлено сообщение (включая отправку с другой сессии самого пользователя); доставка не позднее 2 секунд (NF_Perf_5).
message_edited	{message: Message}	Сообщение отредактировано.
message_deleted	{chat_id, message_id}	Сообщение удалено у всех участников.
message_read	{chat_id, reader_id, last_read_at}	Участник прочитал чат: клиент обновляет статусы «Прочитано» (в личном чате — у получателя; в группе — когда прочитал хотя бы один участник, кроме отправителя).
chat_created	{chat_id}	Пользователь стал участником нового чата: создан групповой чат либо личный чат с ним. Клиент запрашивает GET /chats/{chat_id} и добавляет чат в список (F_Group_1; F_Cont_3, F_Chat_18).
chat_updated	{chat_id}	Изменены название или аватар группы; клиент запрашивает GET /chats/{chat_id} (F_Group_7, 8, 20).

Тип	Содержимое	Когда отправляется
chat_deleted	{chat_id}	Групповой чат удалён владельцем: клиент убирает чат из списка (F_Group_11, 12).
chat_member_added	{chat_id, user_id}	В группу добавлен участник. Получают все участники, включая добавленного; для добавленного клиент запрашивает GET /chats/{chat_id} (F_Group_3, 4).
chat_member_removed	{chat_id, user_id}	Участник удалён из группы или вышел. Получают остальные участники и сам удалённый: его клиент убирает чат из списка (F_Group_9, 10, 13, 14).
chat_role_changed	{chat_id, user_id, role}	Участнику назначена или снята роль admin либо роль owner передана при выходе владельца (F_Group_5, 6, 15, 16).
pong	{}	Ответ на ping.
error	{code, message}	Некорректное сообщение клиента.

Взаимодействие Gateway и сервиса онлайн-статусов. API Gateway держит WebSocket-соединения клиентов и вызывает внутренние эндпоинты Сервиса онлайн-статусов (раздел 4.6): при подключении клиента — client_connected, при отключении — client_disconnected (с user_id и идентификатором соединения), на каждый ping — продление соединения, на subscribe_status и unsubscribe_status — изменение подписок. Сервис онлайн-статусов хранит в Redis ключ на каждое соединение со временем жизни 75 секунд (продлевается каждым ping), индекс соединений по пользователю, отсортированное множество идентификаторов соединений с оценкой, равной моменту истечения ключа, и подписки клиентов (для каждого наблюдаемого пользователя — множество подписанных соединений; при отключении соединения его подписки удаляются). Чтобы доставить событие, сервис вызывает эндпоинт доставки Gateway со списком пар «соединение — сообщение». Пользователь Online, пока у него есть хотя бы одно соединение; status_changed отправляется только при появлении первого и исчезновении последнего соединения. Раз в секунду сервис выбирает из множества соединения, чей момент истечения прошёл, удаляет их и обрабатывает как отключения; уведомления Redis об истечении ключей не используются. Закрытие соединения клиентом Gateway обнаруживает сразу, и статус меняется в пределах 2 секунд (NF_Perf_4). При обрыве сети без закрытия соединения разрыв обнаруживается по истечении 75 секунд без ping, и срок NF_Perf_4 отсчитывается от этого момента. Брокер в этом обмене не участвует.

4.5. События брокера

Сервисы обмениваются событиями через брокер *RabbitMQ* (общий обменник *kokogram.events* типа *topic*, ключ маршрутизации совпадает с типом события). Каждое событие имеет вид **{"event_id": uuid, "type": "...", "occurred_at": время, "payload": {...}}**. Доставка выполняется как минимум один раз, поэтому обработка событий идемпотентна по смыслу: клиент применяет повторно полученное событие сообщения или чата по id сообщения или чата, не меняя результат, а очистка по chat.deleted безопасна при повторе (удаление уже отсутствующих файлов и записей кэша ничего не меняет). Каждый потребитель читает собственную очередь, привязанную к обменнику по ключам нужных ему событий. Отдельное хранилище обработанных event_id не требуется. Если событие потеряно (например, сервис не успел опубликовать его после записи в БД), данные остаются в БД и клиент получает их при следующем запросе: сообщения и список чатов загружаются запросами GET. Потеря chat.deleted оставляет в хранилище лишние файлы, но данные пользователей не затрагивает.

Событие	Публикует	Потребитель	Содержимое (payload)	Назначение
message.created	Сервис сообщений	Сервис онлайн-статусов	{message: Message, recipient_ids: [uuid]}	Доставить новое сообщение клиентам всех участников чата, которые сейчас в сети (NF_Perf_5).
message.edited	Сервис сообщений	Сервис онлайн-статусов	{message: Message, recipient_ids}	Доставить изменённое сообщение.
message.deleted	Сервис сообщений (удаление «у всех»)	Сервис онлайн-статусов	{chat_id, message_id, recipient_ids}	Убрать сообщение у онлайн-участников.
message.read	Сервис сообщений	Сервис онлайн-статусов	{chat_id, reader_id, last_read_at, recipient_ids}	Обновить статусы «Прочитано» у онлайн-участников.
chat.created	Сервис чатов (создание группового чата; создание личного чата)	Сервис онлайн-статусов	{chat_id, recipient_ids}	Сообщить онлайн-участникам о новом чате (WebSocket-сообщение chat_created).
chat.updated	Сервис чатов (изменение названия или аватара группы)	Сервис онлайн-статусов	{chat_id, recipient_ids}	Сообщить онлайн-участникам об изменении группы (chat_updated).
chat.deleted	Сервис чатов (удаление группы)	Сервис онлайн-статусов, Сервис сообщений	{chat_id, recipient_ids}	Убрать чат у онлайн-участников; recipient_ids — участники на момент удаления (chat_deleted). Сервис сообщений по этому событию удаляет файлы чата (префикс chats/{chat_id}/) и его запись в кэше.

Событие	Публикует	Потребитель	Содержимое (payload)	Назначение
chat.member_added	Сервис чатов (добавление участника)	Сервис онлайн-статусов	{chat_id, user_id, recipient_ids}	Сообщить онлайн-участникам, включая добавленного, о новом участнике (chat_member_added).
chat.member_removed	Сервис чатов (удаление участника, выход из группы)	Сервис онлайн-статусов	{chat_id, user_id, recipient_ids}	Сообщить об уходе участника; recipient_ids включают самого удалённого (chat_member_removed).
chat.role_changed	Сервис чатов (назначение и снятие роли admin, передача роли owner при выходе владельца)	Сервис онлайн-статусов	{chat_id, user_id, role, recipient_ids}	Сообщить онлайн-участникам об изменении роли (chat_role_changed).

Список получателей recipient_ids для событий сообщений Сервис сообщений получает от Сервиса чатов при проверке участника, а события чатов публикует сам Сервис чатов, владеющий списком участников. Поэтому Сервису онлайн-статусов не нужно обращаться к данным чатов.

4.6. Внутренние вызовы между сервисами

Синхронные вызовы, которые не доступны клиенту: адреса начинаются с **/internal** и не маршрутизируются через API Gateway наружу. Они соответствуют стрелкам между сервисами на схеме архитектуры.

Вызов	Кто и к кому	Назначение	Ответ
POST /internal/sessions/verify	API Gateway -> Сервис пользователей	Проверить токен: по хэшу найти сессию и убедиться, что срок не истёк.	{user_id} или 401
GET /internal/chats/{chat_id}/membership/{user_id}	Сервис сообщений -> Сервис чатов	Проверка участника: состоит ли пользователь в чате, тип чата, список участников (для recipient_ids) и их last_read_at (для статусов «Прочитано»).	{is_member, chat_type, members: [{user_id, role, last_read_at}]}
PUT /internal/chats/{chat_id}/members/{user_id}/last-read	Сервис сообщений -> Сервис чатов	Записать момент прочтения чата участником (при POST /chats/{chat_id}/read).	204
GET /internal/blocks/check?user_a=&user_b=	Сервис сообщений -> Сервис пользователей	Проверка блокировки в обе стороны; вызывается только для личных чатов.	{blocked}

Вызов	Кто и к кому	Назначение	Ответ
GET /internal/contacts/check?owner_id=&user_id=	Сервис чатов -> Сервис пользователей	Проверка существования пользователя и наличия его в контактах владельца (при добавлении в группу).	{user_exists, is_contact}
PUT /internal/chats/personal	Сервис пользователей -> Сервис чатов	Создать личный чат между двумя пользователями или вернуть существующий (при POST /contacts). Тело: {user_a, user_b}. Операция идемпотентна благодаря уникальности direct_key; для вновь созданного чата сервис чатов публикует chat.created. При обработке сервис чатов не обращается к сервису пользователей, поэтому взаимной блокировки вызовов нет.	{chat_id, created}
GET /internal/users?ids=	Сервис чатов -> Сервис пользователей	Получить профили пользователей по списку идентификаторов (не более 100): для участников и собеседников в ответах API и для выбора нового владельца по username.	{items: [{id, username, status, avatar_url}]}, avatar_url — подписанная ссылка
GET /internal/gifs/check?url=	Сервис сообщений -> Сервис GIF	Проверить адрес GIF перед сохранением вложения: хост входит в список доменов CDN Giphy (https), размер не более 50 МБ (NF_Gif_3). Если CDN недоступен, возвращается gif_unavailable.	{valid, reason}, reason — null, host_not_allowed или file_too_large
POST /internal/presence/connections	API Gateway -> Сервис онлайн-статусов	Клиент подключился (client_connected). Тело: {connection_id, user_id}. Создаёт ключ соединения; если оно первое у пользователя, статус становится Online, и подписчикам уходит status_changed.	204
DELETE /internal/presence/connections/{connection_id}	API Gateway -> Сервис онлайн-статусов	Клиент отключился (client_disconnected): удаляются ключ и подписки соединения; если оно было последним у пользователя, статус становится Offline.	204
POST /internal/presence/connections/{connection_id}/ping	API Gateway -> Сервис онлайн-статусов	Продлить время жизни соединения на 75 секунд (на каждый ping клиента).	204 или 404 (соединение неизвестно)
PUT /internal/presence/connections/{connection_id}/subscriptions	API Gateway -> Сервис онлайн-статусов	Изменить подписки на статусы (subscribe_status, unsubscribe_status). Тело: {add: [uuid], remove: [uuid]}.	{statuses: [{user_id, online}]} для add; Gateway отправляет клиенту status_snapshot

Вызов	Кто и к кому	Назначение	Ответ
POST /internal/ws/deliver	Сервис онлайн-статусов -> API Gateway	Доставить сообщения в WebSocket-соединения. Тело: {deliveries: [{connection_id, type, payload}]}; type и payload — как в разделе 4.4.	204

Маршрутизация Gateway. Gateway проверяет токен и направляет запрос по префиксу пути: **/auth**, **/users**, **/contacts** и **/blocks** — в Сервис пользователей; **/chats** (кроме **/chats/{id}/messages** и **/chats/{id}/read**) — в Сервис чатов; **/chats/{id}/messages**, **/chats/{id}/read** и **/messages** — в Сервис сообщений; **/gifs** — в Сервис GIF; **/ws** обслуживает сам Gateway. Шифрование трафика также обеспечивает Gateway.

5. Тестируемость

Раздел описывает решения, принятые в Kokogram, чтобы систему можно было покрыть модульными, интеграционными и системными тестами без переписывания кода. Внутренняя структура каждого модуля показана на диаграммах модулей: слева находятся компоненты, которые обращаются к модулю, справа — компоненты, к которым обращается он сам, в центре — слои самого модуля.

5.1. Границы модулей

Модулями считаются шесть компонентов: API Gateway и пять сервисов (пользователей, чатов, сообщений, онлайн-статусов и GIF). Каждый модуль владеет своими данными и взаимодействует с остальными только через опубликованные интерфейсы: REST, WebSocket, события брокера и внутренние вызовы /internal. Внутри каждый модуль состоит из четырёх слоёв.

Входящий слой — роутеры FastAPI, обработчики событий брокера и WebSocket-эндпоинт. Он только разбирает запрос, валидирует его схемой Pydantic, вызывает один метод сервиса и преобразует результат или исключение в ответ с кодом из таблицы пункта 4.1. Бизнес-логики во входящем слое нет.

Бизнес-логика — классы-сервисы. В них находятся все правила FS: проверки, вычисления, ветвления, роли, лимиты. Сервис — обычный класс Python, который не импортирует FastAPI, SQLAlchemy, httpx, клиенты Redis и RabbitMQ; всё внешнее он получает через конструктор в виде портов.

Порты — интерфейсы (typing.Protocol), описывающие, что сервису нужно от внешнего мира, в терминах предметной области (найти пользователя по e-mail, опубликовать событие, сохранить объект по ключу), а не в терминах технологий.

Адаптеры — реализации портов на конкретных технологиях. Это единственный слой, который знает о SQL, Redis, AMQP, S3 и HTTP.

Правила, которым не нужен внешний мир, выделены в чистые компоненты без зависимостей: Validators (сервис пользователей), RolePolicy (сервис чатов), MessagePolicy (сервис сообщений), GifVariantSelector (сервис GIF) и RouteTable (API Gateway). Их проверяют

модульные тесты без заглушек, перебирая входные значения (классы эквивалентности, граничные значения, таблицы решений). Границы модулей и их содержимое сведены в таблицу.

Модуль	Входные интерфейсы	Логика внутри модуля	Выходные зависимости
API Gateway	**REST /api/v1/*; WebSocket /ws; POST /internal/ws/deliver**	Проверка токена; маршрутизация по префиксу пути; реестр WebSocket-соединений; протокол канала. Бизнес-логики нет	Сервисы пользователей, чатов, сообщений, GIF и онлайн-статусов (HTTP)
Сервис пользователей	**/auth, /users, /contacts, /blocks; /internal/sessions/verify, /internal/blocks/check, /internal/contacts/check, /internal/users**	Регистрация и вход; нормализация e-mail и телефона; сессии (срок 30 суток); профиль; контакты (лимит 50); блокировки	PostgreSQL; SeaweedFS (аватары); сервис чатов (HTTP)
Сервис чатов	**/chats*; /internal/chats/personal, /internal/chats/{id}/membership, /last-read**	Личные и групповые чаты; direct_key; участники; роли owner, admin, member и передача owner	PostgreSQL; SeaweedFS (аватары групп); RabbitMQ (публикация chat.*); сервис пользователей (HTTP)
Сервис сообщений	**/chats/{id}/messages, /read, /messages/*; событие chat.deleted**	Отправка, ответы; окно редактирования 24 ч; удаление у всех и у себя; вложения; история с курсором; статусы «Отправлено» и «Прочитано»	PostgreSQL; Redis; SeaweedFS; RabbitMQ (публикация message.*); сервисы чатов, пользователей и GIF (HTTP)
Сервис онлайн-статусов	**/internal/presence/*; события message.* и chat.***	Учёт соединений; время жизни 75 с; статусы Online и Offline; подписки; определение получателей события	Redis; API Gateway (POST /internal/ws/deliver)
Сервис GIF	**/gifs/search, /gifs/trending; /internal/gifs/check**	Поиск и тренды; выбор вариантов файла по размеру; проверка адреса GIF (домен и размер); кэширование	Giphy API; CDN Giphy; Redis

5.2. Точки подмены зависимостей

Сервис не создаёт свои зависимости сам, а получает их через конструктор. Выбор реализации выполняется в одном месте модуля — в функциях зависимостей FastAPI (Depends). В модульных тестах заглушки передаются в конструктор сервиса напрямую, в тестах с HTTP-запросами подставляются через app.dependency_overrides. Реализация порта и заглушка проверяются одним набором контрактных тестов:

он запускается и на заглушке, и на реальном адаптере в тестовом окружении, поэтому заглушка ведёт себя так же, как настоящая реализация.

Подменяемые абстракции сведены в таблицу.

Порт	Модули	Реализация	Заглушка в тестах	Что позволяет проверить
Репозитории: UserRepository, SessionRepository, ContactRepository, BlockRepository, ChatRepository, MemberRepository, MessageRepository	Пользователей, чатов, сообщений	SQLAlchemy, PostgreSQL	In-memory репозиторий на словарях с теми же ограничениями уникальности (e-mail, телефон, username, direct_key)	Логику сервисов без БД
Кэш и хранилище состояния: RecentMessagesCache, GifCache, PresenceStore	Сообщений, GIF, онлайн-статусов	Redis	In-memory реализация, время жизни записей которой считается по Clock	Время жизни и сброс записей кэша, срок 75 с без реального ожидания
Файловое хранилище: FileStorage, AvatarStorage	Пользователей, чатов, сообщений	S3-клиент, SeaweedFS	Словарь «ключ объекта -> байты»	Загрузку, удаление по префиксу, ограничения размера
EventPublisher	Чатов, сообщений	RabbitMQ, обменник kokogram.events	RecordingPublisher: складывает события в список; может имитировать отказ брокера	Какие события и с каким recipient_ids опубликованы; поведение при недоступности брокера
Обработчики входящих событий	Сообщений, онлайн-статусов	Подписка на очередь RabbitMQ	Обработчик — обычная функция от словаря события; тест вызывает её напрямую	Обработку chat.deleted, message.* и chat.*; идемпотентность повторной доставки (раздел 4.5)
Межсервисные клиенты: UsersClient, ChatsClient, GifClient, PresenceClient, GatewayClient, UpstreamClient	Все модули	httpx, вызовы /internal	Клиент-заглушка с заданными ответами или httpx.MockTransport	Ответы 200, 403, 404 соседнего сервиса и его недоступность (service_unavailable, таймаут)
Внешний GIF-каталог: GiphyClient, CdnClient	Сервис GIF	httpx: Giphy API, CDN Giphy	Заглушка с фиксированными ответами; в окружении — контейнер giphy-stub	gif_unavailable (F_Gif_7); границы 1 МБ и 50 МБ (NF_Gif_2, NF_Gif_3) без реальной сети
PasswordHasher	Сервис пользователей	bcrypt	Быстрая заглушка	Ускоряет тесты; хранение именно хэша (NF_Sec_2) проверяется на реальной реализации

5.3. Управление состоянием и временем

Сервисы не вызывают datetime.now(), uuid4(), random, secrets и time.sleep напрямую. Это проверяется при ревью pull request и статическим анализом (ruff, правила DTZ для работы с датой и временем). Абстракции сведены в таблицу.

Источник	Абстракция	В продакшене	В тестах	Где используется и что проверяется
Текущее время	Clock.now() — время в UTC	SystemClock	FakeClock: время задаётся и сдвигается вручную (advance)	Окно редактирования 24 ч (F_Chat_10); срок сессии 30 суток (NF_Sec_3); срок подписанных ссылок 15 мин; время жизни соединения 75 с; значения created_at, edited_at, last_read_at
Идентификаторы	IdGenerator.new_id()	uuid4	Последовательные предсказуемые UUID	Ожидаемые значения в ответах; порядок сообщений в курсорной выдаче
Токены сессий	TokenGenerator.new_token()	secrets.token_urlsafe	Фиксированные значения	Проверка, что в хранилище попадает только хэш токена (NF_Sec_4)
Подписанные ссылки	UrlSigner (HMAC-SHA256, Clock)	Секрет из настроек	Тестовый секрет и FakeClock	Корректность подписи и истечение ссылки (раздел 1.5)
Фоновые задачи	ExpirationWatcher.run_once()	Цикл раз в секунду вызывает run_once()	Тест вызывает run_once() после FakeClock.advance(76 с)	Просроченные соединения становятся Offline без реального ожидания 75 секунд
Настройки	Settings	Переменные окружения	Объект Settings, созданный в тесте	Лимиты, TTL, адреса сервисов, список доменов CDN Giphy: значение меняется без перезапуска

Состояние. Сервисы не хранят состояние между запросами в памяти процесса: всё состояние находится в PostgreSQL, Redis и объектном хранилище. Поэтому экземпляр сервиса создаётся заново в каждом тесте, и тесты не влияют друг на друга. Единственный объект с состоянием в памяти — реестр WebSocket-соединений API Gateway (ConnectionRegistry), он тоже создаётся отдельно для каждого теста. Тестовые данные каждый тест создаёт сам, с уникальными username, e-mail и телефоном, и не зависит от порядка запуска.

Требования NF_Perf_* и NF_Gif_1 измеряются на реальных часах в полном окружении (раздел 5.4); FakeClock для них не применяется.

5.4. Тестовое окружение

Работоспособная копия системы поднимается одной командой Docker Compose (файл docker-compose.test.yml). Адреса, секреты и лимиты задаются переменными окружения, поэтому в тестах используются те же образы, что и в рабочей системе. В файле два профиля: infra (только инфраструктура и заглушка внешнего сервиса) и full (вся система). Состав контейнеров приведён в таблице.

Контейнер	Назначение	Профиль infra	Профиль full	Замена внешней системы
postgres (версия 15 и выше)	Основная БД; схема создаётся миграциями при старте	да	да	—
redis	Кэш, онлайн-статусы, подписки	да	да	—
rabbitmq	Брокер событий, обменник kokogram.events	да	да	—
seaweedfs	Объектное хранилище с S3-совместимым API	да	да	—
giphy-stub	Заглушка Giphy API и CDN: отдаёт фиксированные GIF; значение Content-Length задаётся в тестовых данных (в том числе 50 МБ и 50 МБ + 1 байт без настоящих файлов); режим отказа включается запросом	да	да	Giphy API и CDN Giphy: нет лимита запросов, нет зависимости от сети, можно смоделировать недоступность
users, chats, messages, presence, gif, gateway	Сервисы Kokogram	нет	да	—
web	Статический сервер веб-клиента	нет	да	—

На разных уровнях тестирования поднимается разный набор компонентов.

Уровень	Что поднимается	Что заменено заглушками	Запуск
Модульные	Ничего, Docker не нужен	Все порты (таблица 5.2), Clock, IdGenerator	pytest, автоматически при сборке
Интеграционные	Профиль infra. Тестируемые модули запускаются в процессе теста как ASGI-приложения и вызывают друг друга без сети (httpx.ASGITransport)	Модули, не участвующие в сценарии, — клиентами-заглушками; Giphy — контейнером giphy-stub; недоступность зависимости моделируется отказом заглушки	pytest -m integration на сервере непрерывной интеграции при изменении кода или тестов
Системные (E2E)	Профиль full: все контейнеры, веб-клиент	Только внешний GIF-сервис (giphy-stub)	Playwright через браузер; запуск вручную или по расписанию

Тестовые данные. Данные создают сами тесты с помощью фабрик (через API или напрямую через сервисы); общего дампа БД нет. Между тестами окружение очищается: таблицы БД очищаются командой TRUNCATE, в Redis выполняется FLUSHDB, очереди RabbitMQ очищаются, объекты SeaweedFS удаляются по префиксу (ключи chats/{chat_id}/ и avatars/ построены так, чтобы это было возможно, раздел 1.5).

Сценарии отказа. Недоступность зависимости моделируется остановкой контейнера (docker compose stop) или переводом giphy-stub в режим отказа. Так проверяются коды service_unavailable и gif_unavailable. Каждый контейнер имеет проверку готовности (healthcheck, раздел 5.5), и команда docker compose up --wait возвращает управление, когда окружение готово к тестам.

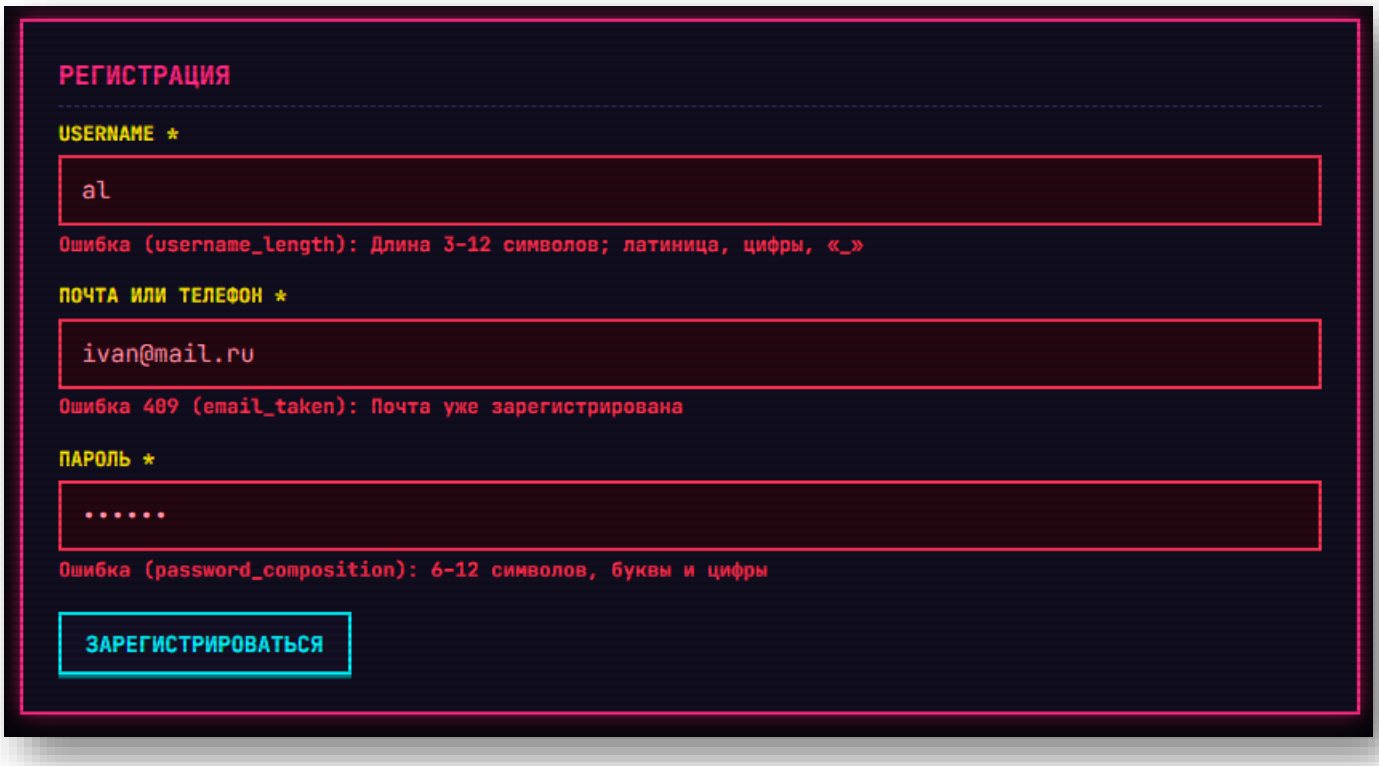
5.5. Наблюдаемость

Чтобы определить результат операции снаружи, не обращаясь к коду, в системе предусмотрены следующие средства.

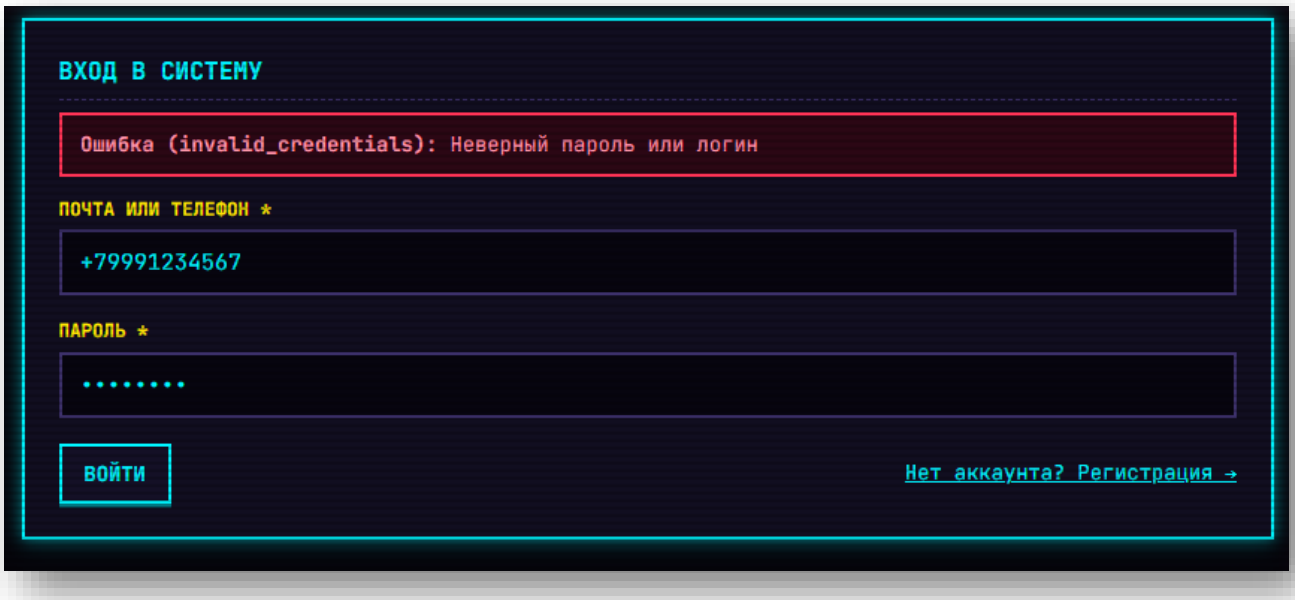
Что проверяется	Как наблюдается извне	Примеры
Результат REST-операции	HTTP-статус и поле error.code в теле ответа (раздел 4.1). Коды ошибок стабильны; текст message тестами не проверяется	409 email_taken (F_Auth_4); 403 edit_window_expired (F_Chat_10)
Состояние хранилищ	Тест выполняет запросы только на чтение: SELECT в PostgreSQL, чтение ключей Redis вместе со временем жизни, поиск объектов SeaweedFS по префиксу	В таблице users хранится хэш пароля, а не пароль (NF_Sec_2); после удаления чата объектов с префиксом chats/{chat_id}/ нет
События брокера	Тест создаёт временную очередь, привязанную к обменнику kokogram.events с ключом #, и читает опубликованные события	message.created содержит в recipient_ids всех участников чата; при удалении группы опубликовано chat.deleted
События в реальном времени	Тестовый WebSocket-клиент подключается к /ws и ждёт сообщение нужного типа с таймаутом	Онлайн-получатель получает сообщение не позднее чем через 2 с (NF_Perf_5); приходит status_changed (NF_Perf_4)
Логи	Каждый сервис пишет в stdout JSON-логи: время, сервис, уровень, событие, request_id, user_id, код ошибки. API Gateway создаёт заголовок X-Request-ID и передаёт его во внутренние вызовы; при обработке события брокера в лог пишется event_id	По request_id в логах восстанавливается цепочка вызовов между сервисами; повторная доставка события отмечается в логе
Готовность сервиса	GET /health у каждого сервиса: 200, если сервис и его зависимости (БД, Redis, брокер, хранилище) доступны. Адрес не маршрутизируется через API Gateway	Используется в docker compose up --wait и на сервере непрерывной интеграции
Время отклика	Тест измеряет интервал между запросом и ответом на реальных часах	Авторизация не дольше 500 мс (NF_Perf_2); поиск GIF не дольше 1,5 с (NF_Gif_1)

6. Макет интерфейса

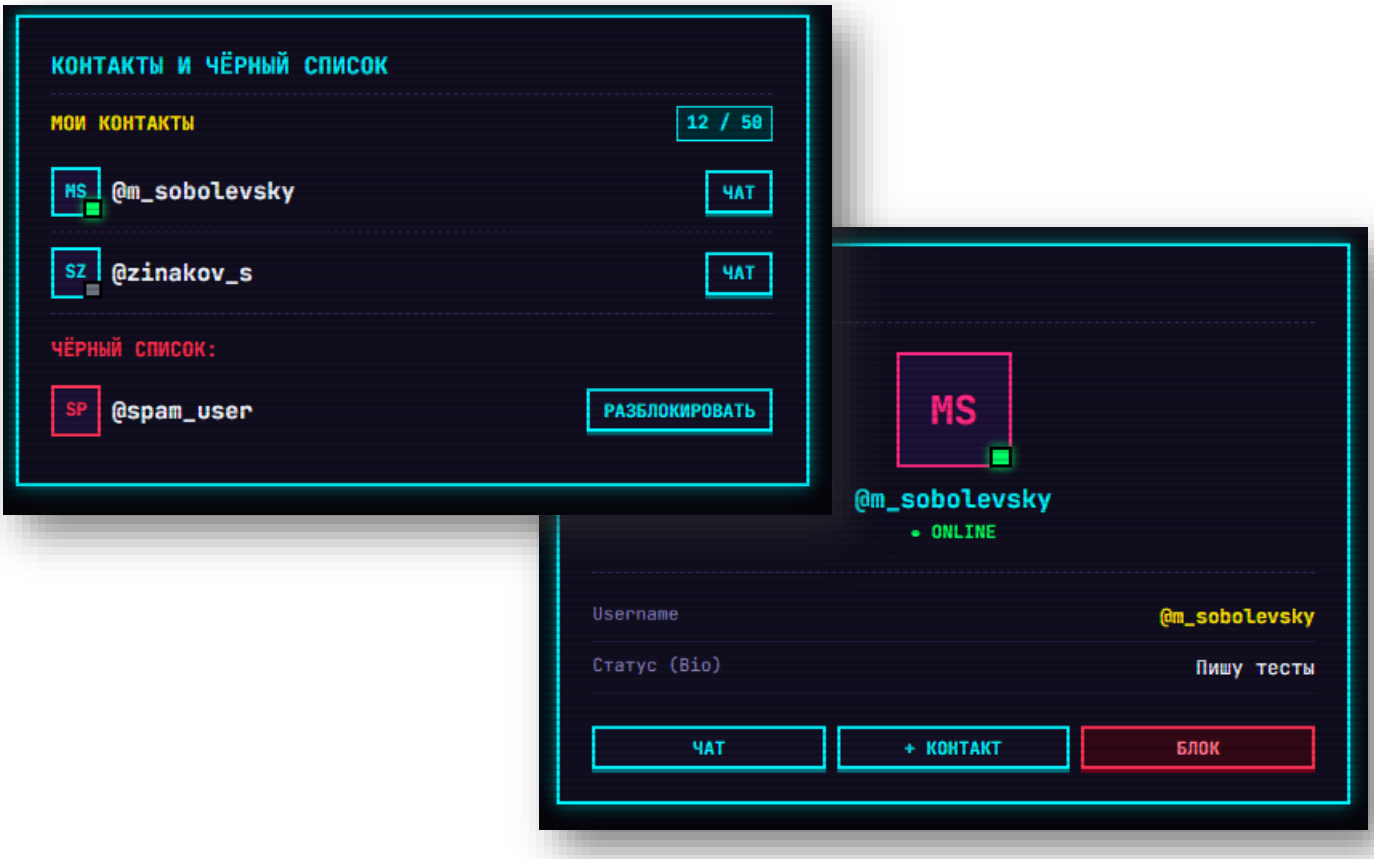
Регистрация



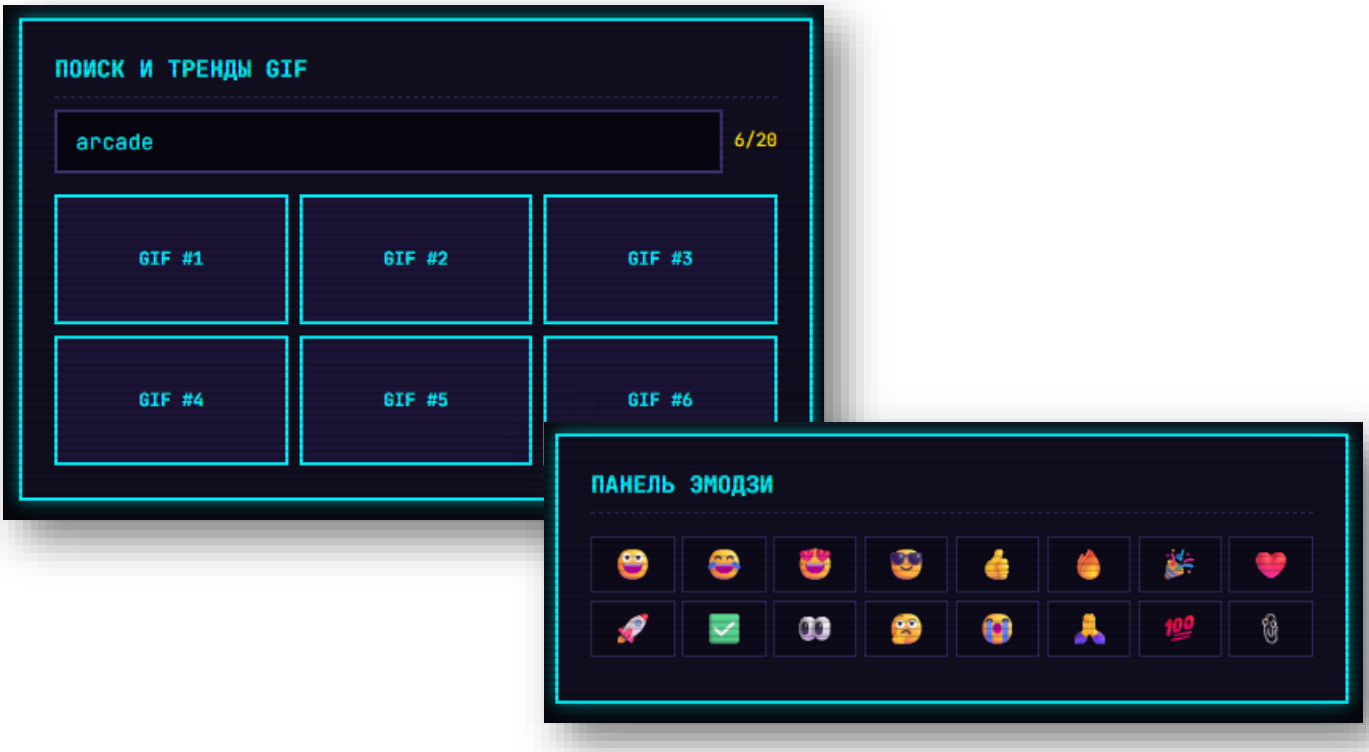
Вход



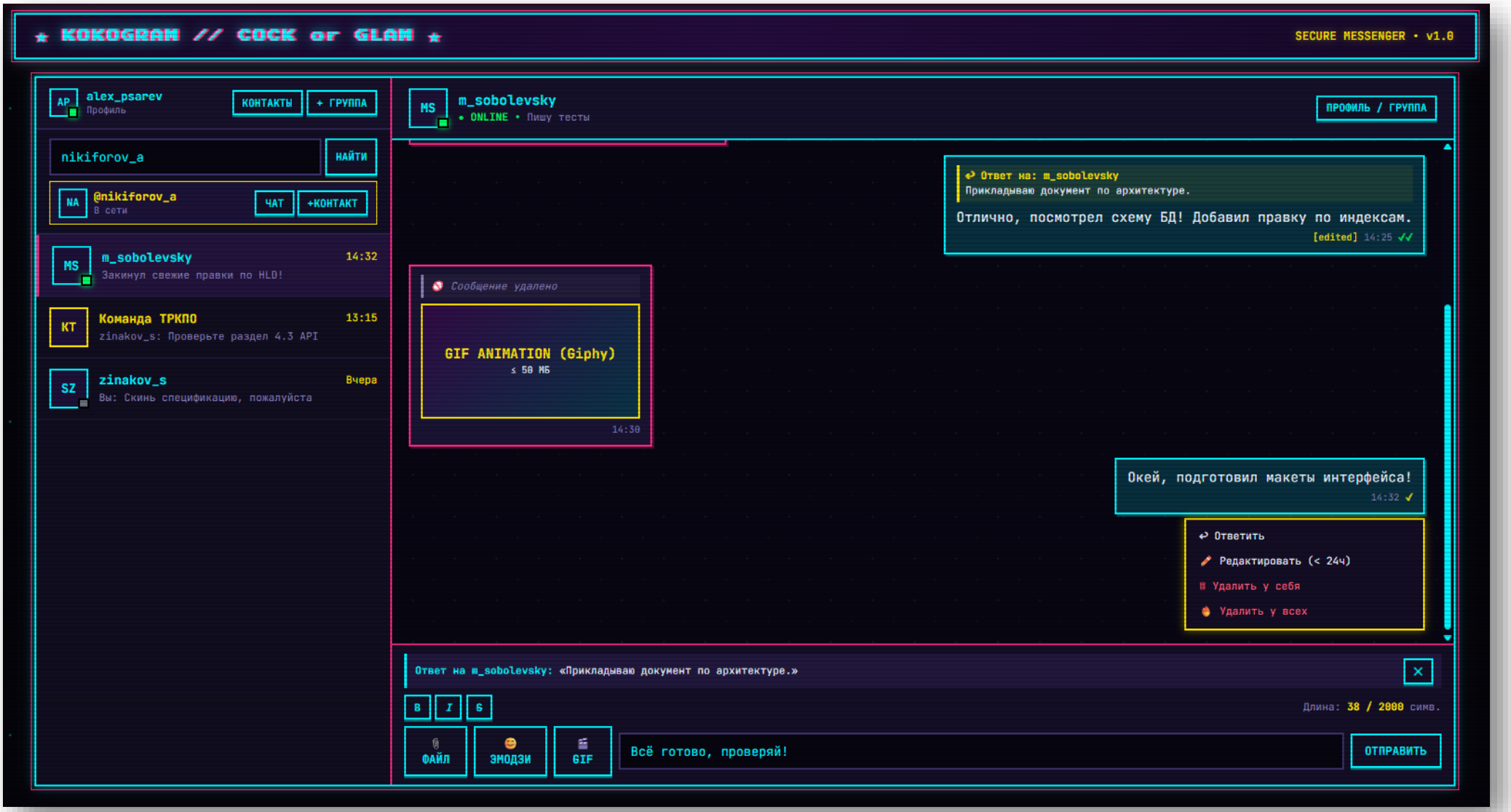
Контакты/Профиль другого юзера



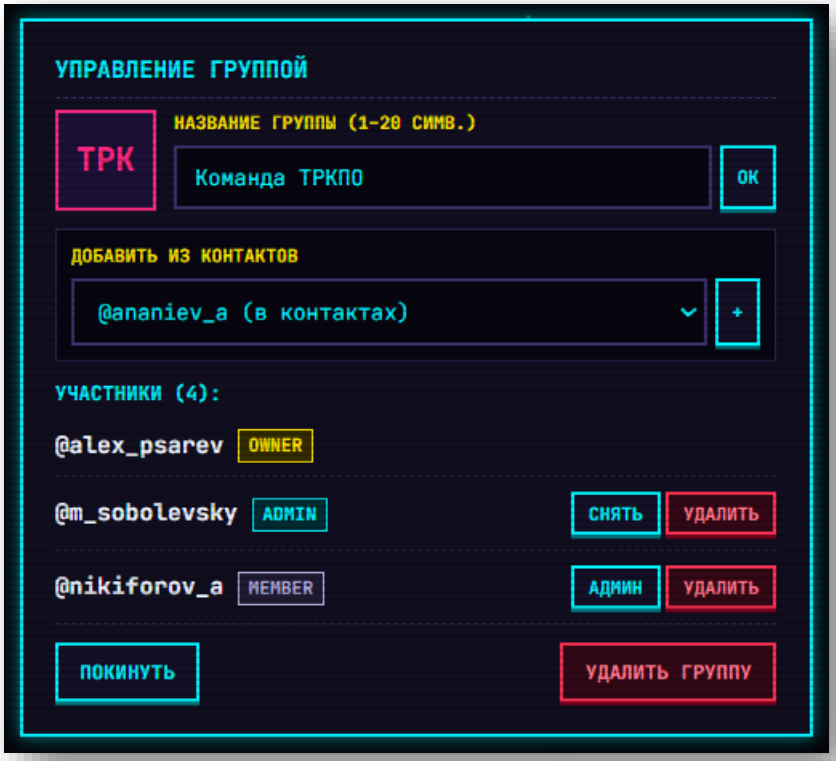
Панели выбора GIF/эмодзи



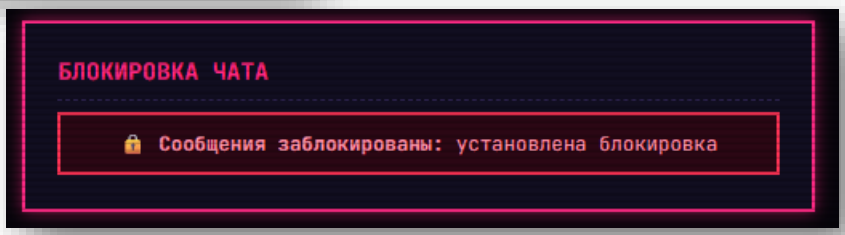
Главная панель (с открытым чатом)



Управление группой



Отображение блокировки



Свой профиль

